

Chapter 1

Introduction

1.1 Background and Motivation

Almost everything people and businesses do together runs on contracts, but actually reading them is a luxury. A typical agreement in our dataset has a median length of about 33,000 characters roughly 5,000 words and the longest one crosses 338,000 characters. Somewhere inside all that text sit the provisions that really matter: who owns newly created intellectual property, whether liability has a ceiling, whether the contract quietly renews itself, whether one party is banned from competing after it ends. Getting a lawyer to review such a document takes hours and costs money that individuals and small businesses often do not have. So people sign contracts they have not fully understood, and carry risks they never saw.

The Contract Understanding Atticus Dataset (CUAD) [1] was built to help automate this kind of review. It contains 510 real commercial contracts in which legal experts highlighted every clause belonging to 41 categories that matter in corporate transactions. The annotation effort alone is said to be worth around two million US dollars of attorney time. But CUAD frames the problem purely as extraction: given a category, find the clause text. Once it finds a clause labelled “Uncapped Liability,” it stops. There is no warning about what that clause means for the signer, no plain-language explanation, and no check of how clauses interact.

Our project tries to close that gap. On top of clause detection and extraction, we add a prototype taxonomy that assigns every *category* a High, Medium or Low level from the weaker party’s point of view, and a text-generation component intended to render each clause as a single plain-English sentence. We packaged the whole pipeline into *Clausify*, a prototype web application where a user pastes or uploads a contract and gets back a risk-prioritized clause report. Both added layers turned out to be weaker than we expected the taxonomy is unvalidated (Section 4.5.4) and the generation component learned to emit category templates rather

than clause-specific summaries (Section 6.5) and Chapters 6 and 8 report that rather than the intention.

1.2 System at a Glance

Before the detailed treatment in Chapter 4, Figure 1.1 gives the shape of the whole system in one picture. The system answers three questions about a contract in order *what is in it*, *where is it*, and *what does it mean* and only the first two are what CUAD was built to benchmark. The third stage, shaded, is what this thesis adds.

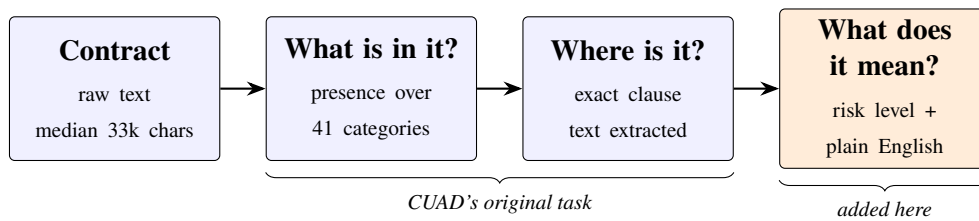


Figure 1.1: The three questions the pipeline answers about a contract.

1.3 Problem Statement

Concretely, the thesis works on four connected problems:

1. **Presence classification.** For each of the 41 CUAD categories, decide whether a given contract contains at least one clause of that category. The main technical obstacle here is length: contracts are roughly 16× longer than the input window of a standard transformer encoder.
2. **Span extraction.** For every category judged present, locate the exact clause text inside the contract. We formulate this as extractive question answering in the style of SQUAD[8].
3. **Plain-language summarization.** Rewrite each extracted clause as one sentence that a non-lawyer can understand.

4. **Risk communication.** Attach a category-level risk level and a one-line reason to every detected clause, so the output reads as a prioritized report rather than a list of quotations. Whether this is genuinely useful to a signer is a usability question we did not evaluate (Section 8.2).

1.4 Research Questions

The objectives below operationalize five questions, which the rest of the thesis is organized to answer. We state them here with the answer we arrived at, so a reader can check each claim against the evidence rather than hunt for it.

RQ1 *Can a sliding-window transformer outperform a full-document lexical baseline at clause-presence classification on long contracts?*

No. The baseline wins by +0.081 micro-F1, and the gap survives training on the full window set (Section 6.1.4) and three random seeds (Section 6.1.3). Answered in Chapter 6.

RQ2 *How accurately can the system extract clause spans from long contracts?*

Token-F1 0.763 (95% CI [0.740, 0.782]) when handed a window containing the clause, but 0.386 when the window is chosen by the presence stage (Sections 6.4 and 6.6).

RQ3 *Does the summarization component produce genuinely clause-specific plain-English output?*

No. It reproduces one of 41 pre-written category templates verbatim on every input; against clause-specific references it scores no better than the template with no model at all (Section 6.5).

RQ4 *How do errors propagate across the complete pipeline?*

Multiplicatively and worse than the components suggest: 21.7% of gold clauses survive all three stages against the 64% the component scores predict (Section 6.6).

RQ5 *Does category-level risk prioritization align the system's errors with what would harm a signer?*

No, and it currently works against it. The configuration we report as the headline misses 37.5% of High-risk clauses, while the component we describe as weaker misses 15.9% (Section 6.3). Aggregate micro-F1 and user harm point in different directions.

Three of these five answers are negative. We report them as the results of the thesis rather than as shortcomings of it: each is a measurement that a reader of the component scores alone would have got wrong.

1.5 Research Objectives

- Build and honestly evaluate a full three-model CUAD pipeline on consumer hardware, using an 80/20 contract-level train/test split so that every reported number comes from genuinely unseen contracts.
- Check whether a windowed transformer actually beats a strong classical baseline for document-level presence classification, instead of assuming that it does.
- Measure span-extraction quality with token-level F1 and exact match on the held-out test set.
- Evaluate the summarizer against two different reference sets, because we found a single automatic metric measures its reference set rather than the task.
- Measure what survives the pipeline end to end, rather than reporting only stage-wise scores.
- Deploy the trained models in a working demonstration application with a risk-prioritized interface.

1.6 Contributions

The main contributions of this work are:

1. **A prototype category-level risk taxonomy for CUAD.** A High/Medium/Low level with a written justification for each of the 41 categories, scored from the weaker party’s perspective (8 High, 22 Medium, 11 Low), together with explicit assignment criteria (Section 4.5.1). We present it as an *initial framework intended for expert validation*, not as a validated legal-risk model: it was authored by the three of us, none of whom has legal training, and as of this writing no legal professional has reviewed it (Section 4.5.4). It attaches risk to a *category*, not to the wording of a particular clause, and we call it category-level risk prioritization throughout for that reason.
2. **An empirical result on long-document presence classification, with its alternative explanations tested.** A full-document TF-IDF baseline (micro-F1 0.775) turned out to be hard to beat. Our windowed DistilBERT loses to it (0.694), and we checked the two obvious excuses: retraining on all 53,662 window examples instead of 24,000 changed the score by -0.0004 , and across three random seeds the gap is 9.5 standard deviations wide. An AND-ensemble nominally finishes first (0.776), but both a paired cluster bootstrap and the ensemble’s own seed variance show that margin is indistinguishable from zero, so we report the ensemble for its error profile rather than its rank.
3. **An end-to-end measurement of error compounding.** Evaluating the three stages in series rather than in isolation shows that only 21.7% of gold clauses survive the whole pipeline, against the roughly 64% the component scores predict, and localizes the loss to a window-selection failure and a train/inference mismatch, both fixable without a larger model.
4. **A demonstration that a small template set turns summarization into classification.** Evaluating the same 150 held-out clauses against our 41 category templates and against clause-specific references drops ROUGE-L from 0.775 to 0.116, and the fine-tuned model is shown to emit a template verbatim on 100% of inputs.
5. **A methodological guardrail: measure the ceiling before training.** We show that question-guided retrieval windowing caps attainable recall at 64%, and we used that one measurement to reject the whole retrieval approach before spending any training time on

it.

6. **A reproducible, laptop-scale pipeline.** All three models train on a single Apple-Silicon laptop. The split, the trained weights, and a standalone evaluation notebook are all saved, so every number in this thesis can be regenerated from disk.
7. **Clausify**, a prototype web application that runs the trained models live and presents the results grouped by risk. It is a demonstration of the pipeline, not an evaluated tool: no usability study was conducted, and Section 7.8 records what it does not do well.

1.7 Thesis Organization

Chapter 2 reviews the related literature. Chapter 3 describes the dataset, the preprocessing, and the exploratory analysis that shaped our design choices. Chapter 4 presents the methodology for all pipeline stages. Chapter 5 covers model training, and Chapter 6 reports the held-out test evaluation, including the confusion-matrix analysis. Chapter 7 describes the Clausify application. Chapter 8 concludes and discusses future work. The appendices contain the full 41-category risk taxonomy and the repository layout.

Chapter 2

Literature Review

2.1 Legal NLP and the CUAD Benchmark

Applying NLP to legal text is an old ambition, but for a long time progress was blocked by a simple problem: legal annotation needs lawyers, and lawyers are expensive. CUAD [1], released at the NeurIPS 2021 Datasets and Benchmarks track by The Atticus Project [2], changed this. It provides 510 real commercial contracts, drawn from public SEC filings, in which experienced attorneys highlighted every span belonging to 41 clause categories that matter in corporate transactions governing law, non-compete, liability caps, IP assignment, and so on.

The dataset ships in the SQuAD format [8]: each (contract, category)

pair is posed as a question over the contract text

and the annotated clause spans are the answers, complete with character offsets.

The original CUAD paper treats the task as extractive question answering and reports that performance scales strongly with model size; their best results come from DeBERTa models at the extra-large scale, and even those scores are modest, which says something about how hard legal clause extraction actually is. Two properties of CUAD end up dominating any modelling effort on it. The first is severe class imbalance: some categories appear in nearly every contract, while others show up barely a dozen times in the whole corpus. The second is document length. Contracts are far longer than a transformer’s input limit, so every practical system has to answer the same question first: how does a model with a 512-token window read a 50,000-character contract?

2.2 Transformer Encoders and Model Distillation

The transformer architecture [3] replaced recurrence with self-attention, and BERT-style masked-language-model pre-training [4] established the fine-tuning recipe this thesis depends on: take a model pre-trained on large general corpora, then adapt it to a downstream task with relatively

little labelled data. This transfer-learning property matters a lot here, because 510 contracts is a small dataset by deep-learning standards.

Since our compute budget is a single laptop, we use DistilBERT [5], a distilled six-layer version of BERT that keeps roughly 97% of BERT’s language-understanding ability while being 40% smaller and 60% faster. The published CUAD baselines use much larger models, so our backbone choice trades some headline accuracy for the ability to reproduce everything on ordinary hardware. We point out this scale difference wherever we interpret our results.

2.3 Long-Document Strategies and Multiple-Instance Learning

The usual remedies for the length mismatch come in three families: (i) *retrieval*, where a cheap scorer picks the few windows most likely to hold the answer and only those go to the transformer; (ii) *sliding windows with aggregation*, where the model scores every window and the document-level prediction aggregates the window scores; and (iii) *sparse-attention architectures* such as Longformer, which widen the input window directly.

The third family deserves more than the one-line dismissal it usually gets, because “too expensive” is not on its own a reasoned argument. We did not run a Longformer experiment, so what follows is a design judgement, and we label it as one throughout rather than dressing it up as a finding; the one claim in it that can be checked cheaply on our own hardware the cost claim we did check, and Section 5.7 reports the measurement.

Our reasoning had three parts, and only the third is about cost. **First, and decisively, we judged that it would not actually solve the problem.** Longformer extends the input window to 4,096 tokens. At roughly four characters per token, our median contract of 33,143 characters is about 8,300 tokens and the longest is about 85,000. so a Longformer would still see only half of a typical contract and a twentieth of the largest one. Sparse attention makes the window eight times wider; on this dataset it does not make it wide enough. We would still need windowing and an aggregation rule on top, which is precisely the design we ended up with anyway only with a backbone several times more expensive underneath it. This part of the argument is arithmetic about the dataset rather than an empirical result, but the arithmetic is not in doubt: the length

problem in CUAD is an order of magnitude past what widening attention fixes.

Second, availability. Our whole pipeline is built around distilled backbones so that it trains and serves on one laptop, and we are not aware of a distilled Longformer with the reliability and tooling support that DistilBERT has; adopting sparse attention would have meant giving up the distillation that makes the rest of the project reproducible.

Third, practical cost. We judged fine-tuning a 148.7M-parameter Longformer at 4,096 tokens to be out of reach on a single Apple-Silicon laptop, and we were wary of custom attention kernels on a non-CUDA device after DeBERTa-v3 produced NaN gradients on the MPS backend (Section 5.6). We should flag immediately that when we finally measured this (Section 5.7), the memory half of the judgement turned out to be *wrong* a training step does fit at batch size 1 while the time half turned out to be right by a very large margin. We have left the reasoning as we made it and reported the correction where the measurement is, rather than quietly rewriting the premise to match the result. There is also an inference-side cost: *Clausify* scores every window of an uploaded contract live for all 41 categories, and the interface is only usable if that finishes in seconds. Unlike the first two points, this one is measurable without training anything, so rather than assert it we benchmarked it Section 5.7 gives forward-pass latency and peak training-step memory for Longformer-base at 4,096 tokens on the machine this thesis was built on, next to the DistilBERT configuration we actually used.

We note that on CUDA hardware with a larger memory budget the third objection would weaken and the second might disappear, while the first would still stand. This is therefore a design judgement grounded in one measurement and one piece of arithmetic, not a demonstration that sparse attention cannot work on CUAD a properly trained Longformer with windowing baseline remains a fair thing for a reader to ask for, and we list it in future work.

Sliding windows aggregated with a maximum correspond to the classical *multiple-instance learning* (MIL) setting: the document is a bag of window instances, the bag is positive if any instance is positive, and max-pooling implements exactly that rule. MIL has a well-known weakness that our experiments ended up confirming: false-positive inflation. With dozens of windows per document, the chance that *some* window fires by accident grows with document

length, and precision suffers.

2.4 Text-to-Text Models for Summarization

T5 [6] reframes every NLP task as text-to-text, which fits clause simplification naturally: legal text goes in, a plain sentence comes out. FLAN-T5 [7] instruction-tunes T5 on a large mixture of tasks, so it gives usable zero-shot behaviour from a prompt as simple as “summarize in plain English:”. We evaluate summaries with ROUGE-L [20], which measures the longest common subsequence against a reference. Chapter 6 shows why this metric has to be paired with actually reading the outputs when the reference set is small.

2.5 Legal Risk Assessment and Decision Support

The risk layer is the part of this thesis with the least literature behind it, and that is a gap in the work rather than a gap in the field. Because we are proposing to tell a non-lawyer which clauses to worry about, the obvious question is what basis exists for such a judgement, and the honest answer is that we did not build on an established framework we built a taxonomy ourselves (Section 4.5.1) and are reporting it as a prototype for that reason.

What the field does provide is context for why that is hard. Legal NLP benchmarks have largely operationalized “risk” indirectly, as classification into legally meaningful categories rather than as a severity judgement: LexGLUE [11] assembles tasks such as judgement prediction and statutory classification, and LEGAL-BERT [10] shows that domain-pretrained encoders help on them but none of these tasks asks a model how *dangerous* a provision is for a particular signer. CUAD itself [1] is explicit that its categories are the clauses lawyers look for in due diligence, which encodes a professional judgement about importance without grading it.

That leaves severity as a genuinely open construct, and it is worth being clear about why it resists a purely data-driven treatment. Risk in a contract is relational: the same uncapped indemnity is dangerous to the party granting it and valuable to the party receiving it, so a label is only meaningful once a perspective is fixed which is the problem Section 4.5.2 confronts and re-

solves only by assumption. It is also contextual (a \$50,000 liability cap is trivial to a corporation and ruinous to a sole trader) and jurisdictional (a non-compete that binds in one jurisdiction is void in another). A per-category constant, which is what we implement, deliberately ignores all three.

Two consequences for how our contribution should be read. First, the appropriate comparison for our taxonomy is not a published risk model we are not aware of an established one covering CUAD’s 41 categories but expert judgement, which is exactly the validation Section 4.5.4 says has not been obtained. Second, work on explainability in legal AI argues that decision support in this domain should expose its reasoning rather than emit a score. Our lookup table is unusually well suited to that: every assignment carries a written justification and can be contested line by line (Appendix A). That transparency is the honest strength of the design, and it is a different property from being correct.

2.6 Plain-Language Legal Communication

Chapter 6 finds that our summarizer does not summarize, and the literature relevant to that finding is about *simplification*, not about T5. The distinction matters: summarization compresses, whereas simplification re-expresses the same content in more accessible language and for a contract clause, compression is largely the wrong operation, since the conditions and exceptions that a summary would drop are frequently the parts that determine what the clause does.

The most directly comparable work is Manor and Li [12], which assembles plain-English summaries of contract sections and observes that models trained on them tend toward generic output an early version of the failure our template pilot quantifies. Readability measurement has a much older tradition [19], and it supplies something our evaluation lacks entirely: a way to check that the output is actually easier to read, rather than merely shorter. We report no readability statistic anywhere, which in hindsight is an omission, since the entire claim of the component is accessibility.

The deeper methodological point this literature makes, and which our results illustrate, is that reference-based metrics measure the reference set. With 41 unique targets, ROUGE-L rewards

recognising the category. Section 6.5 turns that into a measurement by varying the reference set and holding everything else fixed.

2.7 Faithfulness and Hallucination in Generated Summaries

For a system that puts generated sentences next to a user’s own contract, factual correctness of the generation is a safety property, not a quality metric. The summarization literature has established that fluent abstractive output is routinely unfaithful to its source: Maynez et al. [13] find that a large fraction of generated summaries contain content not supported by the input, and that ROUGE correlates poorly with faithfulness judgements; Kryściński et al. [14] propose evaluating factual consistency as a task in its own right. Embedding-based metrics such as BERTScore [15] improve on n-gram overlap for semantic similarity but do not test factuality either.

This bears on our system in a way worth stating precisely, because the shape of our failure is unusual. A template-emitting model **cannot hallucinate in the ordinary sense**: it can only produce one of 41 human-written sentences, all of which are factually sound descriptions of their category. In exchange, it fails in the complementary way it is uninformative about the specific clause, and when it selects the wrong template (30% of the time, Section 6.5) the output is fluent, confident and about a different provision entirely. That is arguably a *worse* failure mode for a legal assistant than a hallucinated detail, because it is undetectable from the output alone: the sentence is well formed and internally correct, and only comparison against the clause reveals it does not describe it.

It also means our evaluation is missing the checks this literature calls for. We report ROUGE-L against two reference sets and read outputs by hand. We do not report BERTScore, any entailment-based faithfulness check, or a human judgement of whether the legal meaning survives and for a component that will be shown to non-lawyers, the last of these is the one that matters. Section 8.3 lists it.

2.8 Gap Addressed by This Thesis

Existing CUAD work stops at locating clauses. A signer plausibly needs more than that: which clauses matter, how much, and what they mean in plain language. This thesis attempts that layer a prototype per-category risk taxonomy, category-template plain-English text, and a demonstration application while also re-examining, on a strict held-out evaluation, the common assumption that a transformer must beat classical lexical methods at the document-level presence decision. We should say at the outset that of the three added components, the empirical results support only the first as a working artefact and neither of the other two as a solved problem; what the added layer contributes is largely a measurement of how far it is from working.

Table 2.1 places this work beside the reference points above along the dimensions that separate them. Two differences are worth reading off it directly. The obvious one is scope: prior work ends at the span, whereas the output here is a risk-ranked, plain-English report, which is the part a non-lawyer can actually act on. The less obvious one is in the long-document column. Every system on CUAD has to make a choice about document length, but that choice is normally asserted; here it was measured before any training happened the retrieval ceiling of 64.0% in Section 4.2.2 is what ruled retrieval out and sent us to multiple-instance learning. Note also what the backbone column costs us: our distilled models are roughly an order of magnitude smaller than CUAD’s DeBERTa-v3-XL, so our absolute extraction scores are not comparable to theirs, and we do not present them as such.

Table 2.1: Related work on legal-document NLP.

Work	Dataset	Task	Model	Long-doc method	Risk	Plain lang.	Evaluation
Hendrycks et al. [1]	CUAD (510)	Span extraction	DeBERTa-v3-XL	Full-doc QA, large model	No	No	Held-out; precision @ 80% recall
Chalkidis et al. [10]	Multiple legal corpora	Domain pretraining + classification	LEGAL-BERT	512-token truncation	No	No	Per-task held-out
Chalkidis et al. [11]	LexGLUE (7 tasks)	Legal language understanding benchmark	BERT/Roberta family	Task-dependent	Indirect (task labels)	No	Benchmark, multi-task
Manor & Li [12]	TL;DR-Legal style	Plain-English summarization	Seq2seq	Section-level inputs	No	Yes	ROUGE + manual
Beltagy et al. [9]	Long-doc benchmarks	Architecture	Longformer	Sparse attention, 4,096 tok	No	No	Task benchmarks
Retrieval-QA pipelines (generic)	Varies	Span extraction	Varies	Retrieval + reader (ceiling usually assumed)	No	No	Varies
This thesis	CUAD (510)	Presence + span + category risk + template text	DistilBERT + FLAN-T5-small	MIL sliding window; ceiling measured, sparse attention benchmarked	Category-level (prototype, unvalidated)	Template (not abstractive)	Contract-level held-out; cluster bootstrap; seed, budget, split controls; end-to-end

Note. “Long-doc method” is how each work handles inputs longer than the encoder window; “Evaluation” notes the level at which results are reported.

Chapter 3

Dataset and Exploratory Analysis

3.1 Data Source

All experiments use the official Atticus Project release of CUAD, obtained from the Hugging Face hub ([theatticusproject/cuad](https://huggingface.co/theatticusproject/cuad)). One practical discovery is worth recording here: the repository's default `load_dataset()` configuration exposes only the 511 raw contract PDFs, which cannot be trained on. The annotated data actually lives in two files inside the repository, and each one plays a different role in our pipeline:

- `master_clauses.csv` an 83-column wide table with one row per contract, which we used for exploratory analysis and for building labels;
- `CUAD_v1.json` the SQuAD-format file with the full contract text and character-level answer offsets, which we used for span-extraction training and full-document context.

3.2 Preprocessing: the Wide-to-Long Melt

The CSV is wide: 41 categories $\times$ (a clause-text column plus an answer column), with a filename column in front. Turning it into a long table of (contract, category) rows forced two decisions that were not obvious at first:

Pair columns by position, not by name. The header `Notice Period To Terminate Renewal-Answer` contains an inconsistent space, which silently breaks any string matching between a category column and its answer column. Pairing by column position (clause text at odd indices, answer at even) survives typos like this.

Define presence from the clause-text list, not the Yes/No answer. Each clause-text cell holds a stringified Python list, either `[]` or `['clause', ...]`. We call a category *present* when this list is non-empty. This rule works the same way across the eight entity-style categories

(dates, party names) whose answer column holds a value instead of Yes/No, and it hands us the clause spans as a side effect.

The melt produces $510 \times 41 = 20,910$ labelled (contract, category) rows, of which 32.1% are positive.

3.3 Exploratory Findings

Four properties of the data ended up shaping every design decision that followed:

1. **Severe class imbalance.** Per-category positive rates run from *Document Name* at 100% and *Parties* at 99.8% down to *Source Code Escrow* at 2.5% and *Price Restrictions* at 2.9%. The median category is present in roughly 23% of contracts.
2. **The rare tail is nearly few-shot.** *Source Code Escrow* has only 13 positive contracts in the entire corpus. No model learns categories like that reliably, and per-category scores on this tail are mostly noise. We carry this caveat through the results chapter.
3. **Contracts are long.** Median 33,143 characters (about 5,000 words), maximum 338,211 characters roughly $16\times$ a transformer’s effective input window. This is the central modelling tension of the whole thesis.
4. **Clauses are short.** The median annotated clause is 196 characters (90th percentile 587, 99th percentile 1,330). So a clause fits easily inside a single window; the hard part is finding it inside a very long document.

It is worth being upfront that 510 contracts is the entire dataset. Its annotations reportedly cost about \$2M of attorney time, so it cannot simply be grown. Three consequences follow: transfer learning from pre-trained models is mandatory; overfitting is the main enemy, so all our conclusions rest on held-out numbers; and the train/test boundary has to be drawn at the contract level.

3.4 The 80/20 Train/Test Split

Following the evaluation protocol set by our supervisor, we split the dataset **80/20 at the contract level**: whole contracts go to one side or the other, so no clause from a training contract can leak into the test set. The split is generated with a fixed seed (42) and *reused identically by all three models*, which means “test” always refers to the same 102 unseen contracts. Table 3.1 summarizes the split; the nearly identical positive rates on both sides confirm it is balanced. We exported the two sides as `data/train_80.csv` and `data/test_20.csv` so the split itself is reproducible.

Table 3.1: The contract-level 80/20 train/test split.

	Contracts	(Contract, category) rows	Positive rate
Train (80%)	408	16,728	32.0%
Test (20%)	102	4,182	32.2%
Total	510	20,910	32.1%

The aggregate positive rates in Table 3.1 are almost identical (32.0% vs. 32.2%), but a single pair of percentages can hide a badly skewed split two sides can agree overall while disagreeing sharply category by category. Figure 3.1 therefore compares the two sides for all 41 categories at once. The profiles track each other closely across the whole frequency range (Pearson $r = 0.986$, mean absolute difference 3.4 percentage points). The visible exception is *Termination For Convenience*, which appears in 47.1% of test contracts against 33.1% of training contracts, a 14.0-point gap; the other outliers are all in the rare tail, where a shift of one or two contracts moves the percentage several points on its own. Since the split was drawn once at random and never re-drawn to look better, we report these gaps rather than tune them away but they are worth carrying into Chapter 6, because a category over-represented in the test set carries more weight in the micro-averaged score than its training frequency would suggest.

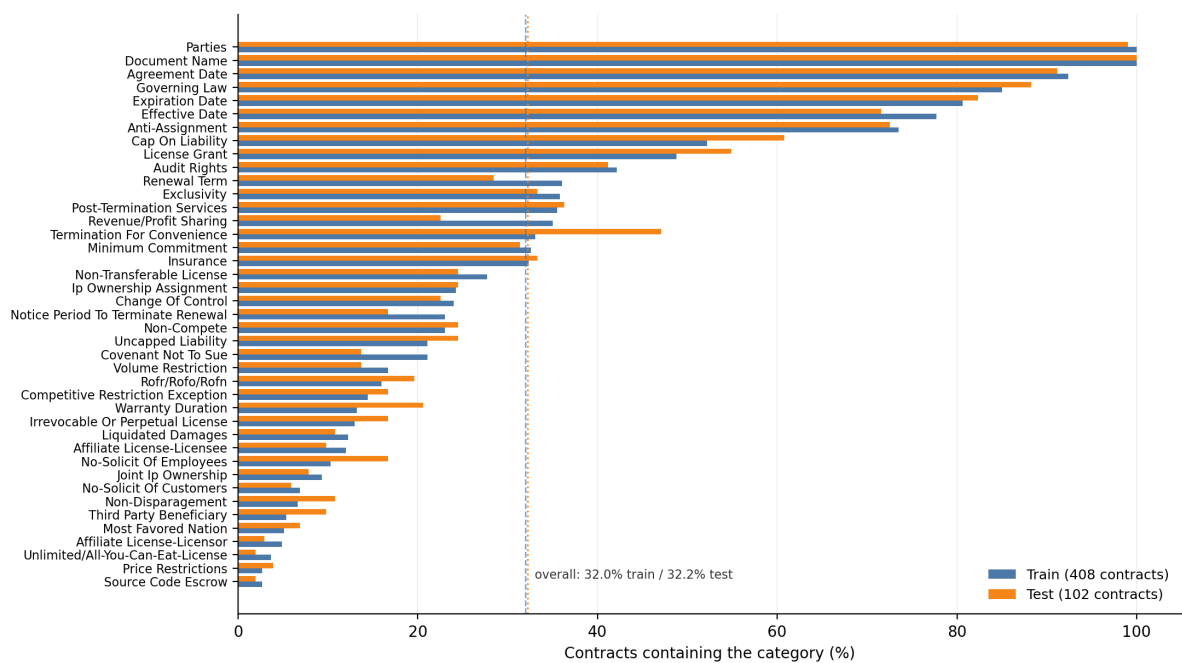


Figure 3.1: Per-category positive rate in the training and test halves.

Chapter 4

Methodology

4.1 Pipeline Overview

The system is a staged pipeline (Figure 4.1). Contract text is windowed; a presence stage decides which of the 41 categories the contract contains; a span stage extracts the exact clause text for each detected category; a risk layer attaches the detected *category*’s High/Medium/Low level; and a text-generation stage attaches a plain-English sentence. The final output is a risk-prioritized report. Section 6.5 shows that the last stage does not in fact summarize the clause in front of it, it selects among 41 pre-written category sentences, so the pipeline is better described as classification-to-template than as summarization. Figure 4.1 names the stages and the models behind them; Figure 4.8 at the end of this chapter repeats the same chain with the concrete data object that crosses each boundary, which is the version to consult when reading the implementation.

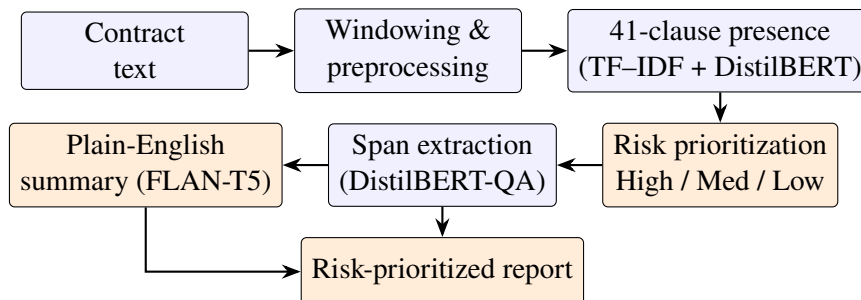


Figure 4.1: The end-to-end pipeline; shaded stages are added by this work.

One principle runs through everything in this chapter: **build a strong, cheap baseline first, and measure feasibility ceilings before spending training time.** Both habits changed the design in concrete ways, as the next sections show.

4.2 Stage 2A: Presence Classification

4.2.1 A strong baseline first: full-document TF-IDF

Classical bag-of-words models have no input-length limit, so a TF-IDF representation gets to read the *entire* contract an advantage no windowed transformer has. We fit a TF-IDF vectorizer (20,000 features, word uni- and bi-grams, sublinear TF, `min_df = 2`) on the 408 training contracts and train one class-balanced logistic regression per category. Legal drafting is extremely formulaic (“*This Agreement shall be governed by the laws of the State of ...*”), so distinctive vocabulary alone carries a lot of signal.

This baseline sets the bar every neural model has to beat.

4.2.2 Measuring the retrieval ceiling and dropping retrieval

The obvious way to hand a long contract to a transformer is retrieval: slide 2,000-character windows with a 1,500-character stride over the text (the windowing scheme itself is drawn in Figure 4.3), score each window by TF-IDF cosine similarity to the category query, and give the model only the best window or windows. The query uses only information available at inference time the category name or its CUAD question, never the answer, so there is no leakage.

Before training anything, we measured the *retrieval hit-rate*: how often does the selected window actually contain the gold clause? Figure 4.2 shows what the measurement is doing and why the answer is a hard bound. Whatever this number is, it is an upper bound on the downstream model’s recall, because a clause sitting in a window the retriever never selects cannot be found by any model placed after it. The best strategy we found (category name as query, top-3 windows) reached only **64.0%** (Table 4.1). Interestingly, the CUAD question text did worse than the bare category name, because the questions share so much boilerplate that all the category queries end up nearly identical. A recall ceiling of 0.64 sits below the baseline’s overall performance (≈ 0.78), so a retrieval-windowed transformer was going to lose before it was ever trained. The measurement took seconds and saved us a wasted training run.

One qualification belongs with that number. What we measured is a ceiling on *lexical* retrieval:

every query strategy in Table 4.1 scores windows by TF-IDF cosine similarity, so a clause phrased without the vocabulary of its category name is invisible to the retriever even when it is plainly present. A dense retriever a sentence encoder embedding both the query and each window matches on meaning rather than on shared terms and would very likely lift the 64.0% figure, possibly above the baseline. We did not test one, so the honest reading of this section is narrower than “retrieval does not work”: it is that *lexical* retrieval imposes a ceiling too low to build on, which was enough to send us to the windowing design in Section 4.2.3. Whether a dense retriever raises the ceiling far enough to make a retrieve-then-read pipeline competitive is an open question we list in future work.

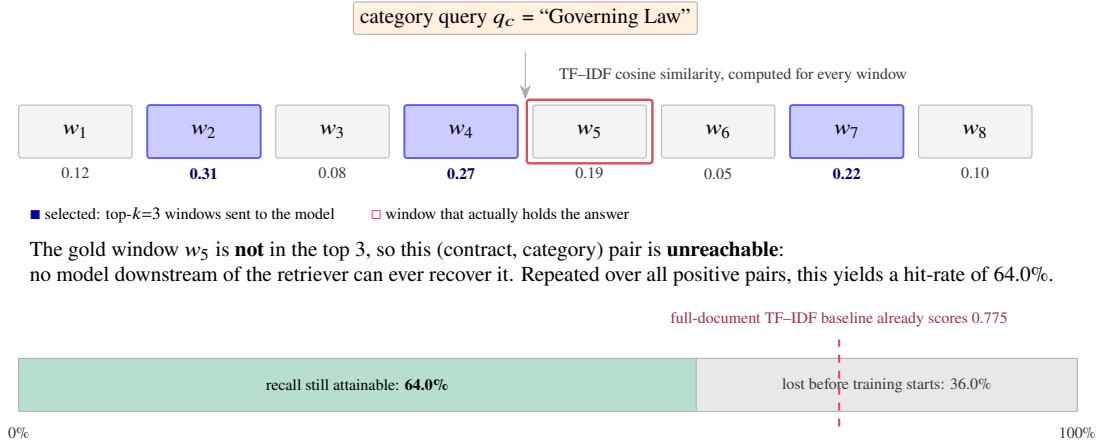


Figure 4.2: Why the measured retrieval ceiling settled the design.

Table 4.1: Retrieval hit-rate on positive (contract, category) pairs.

Query strategy	Top- k	Hit-rate
CUAD question text	1	38.6%
Category name	1	44.6%
Category name	3	64.0%
Name + question	3	61.5%

Note. The hit-rate is the recall ceiling a retrieval-based design would impose.

4.2.3 Sliding-window max-pooling (multiple-instance learning)

To get rid of the ceiling we trained a *window-level* classifier instead: given the pair (category question, window), predict whether that window contains a clause of the category. Figure 4.3 shows how the windows are generated. A 2,000-character window advancing in 1,500-character

steps leaves a 500-character overlap between neighbours, and that overlap is what guarantees no clause can fall between two windows and disappear: the median annotated clause is 196 characters (Section 3.3), comfortably shorter than the overlap, so every clause lands wholly inside at least one window.

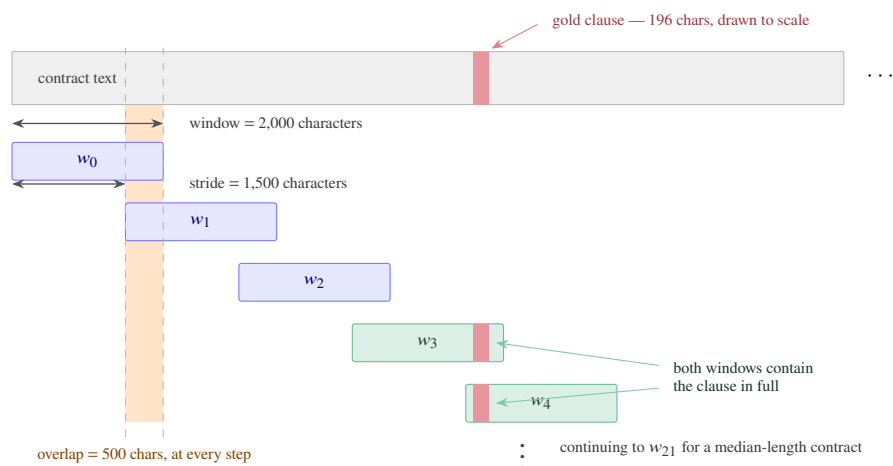


Figure 4.3: Sliding-window generation: 2,000-character windows, 1,500-character stride.

Training labels come from the gold character offsets any window containing an annotated span is positive, and for each (contract, category) bag we sample two negative windows. Using gold offsets this way is ordinary supervision, not leakage: the labels shape the training targets, but they never touch the model’s input, and nothing gold is used at inference. From the 408 training contracts this gives 53,662 window examples (39.2% positive).

At inference the model scores *every* window of the contract, and the document-level prediction is simply the maximum window probability the multiple-instance learning decision rule, drawn in Figure 4.4. The document is a *bag* of window instances; the bag is positive if any instance is positive; and max implements exactly that rule. Since the windows tile the whole document with overlap, every present clause is reachable, so the recall ceiling disappears by construction. The price is precision, and Figure 4.4 also shows why: the maximum is taken over every window, so a single spurious window anywhere in the document is enough to flip the whole contract to positive. With a median of 22 windows per contract and a mean of 35, the chance that some window crosses the 0.5 threshold by accident grows with document length long contracts are structurally more likely to produce a false positive than short ones, independently of what they actually contain.

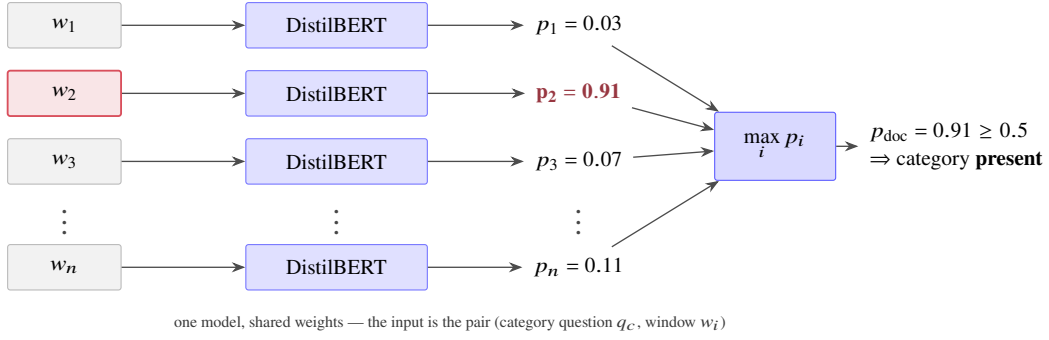
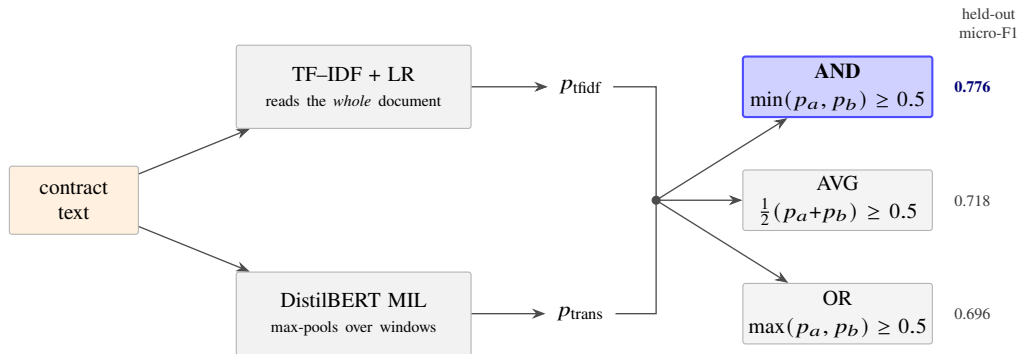


Figure 4.4: The multiple-instance learning rule: maximum over window scores.

4.2.4 The conjunctive ensemble

When we compared the two models category by category, they turned out to be *complementary*: the transformer wins on semantically subtle categories (for example *No-Solicit of Employees* and *Liquidated Damages*), while the baseline wins on keyword-heavy ones. We combine them with **parameter-free** rules AND (min of the two probabilities), OR (max), and AVG (mean) chosen deliberately because they contain nothing tunable, so nothing can overfit to the test set. Figure 4.5 lays out the decision. The AND rule, which reports a clause only when both models agree, filters out each model’s uncorrelated false positives: for a spurious detection to survive, the lexical model and the windowed transformer must both make the same mistake on the same contract and category, and their mistakes are largely independent. That recovers exactly the precision that max-pooling loses.



All three rules are **parameter-free**: there is nothing to fit, so the test set cannot tune them. AND fires only when *both* models cross the threshold, so a false positive has to be made twice, independently, to survive.

Figure 4.5: The conjunctive ensemble and its three parameter-free rules.

4.3 Stage 2B: Span Extraction

Here the transformer has no substitute a bag-of-words model cannot point at character positions. We fine-tune `DistilBertForQuestionAnswering`: the input is (CUAD question, window text) and the model predicts start and end token positions. Character offsets from `CUAD_v1.json` are converted to token labels through the tokenizer’s offset mapping.

Two different window sizes are in play in this thesis, and it is worth being explicit about the difference, since mixing them up makes the whole design look inconsistent. The 2,000-character window of Section 4.2.3 is a *presence* window: it tiles the document blindly, because at that point we do not know where anything is. The window used here is a *focus* window: 1,200 characters, and it is not tiled at all but re-centred on the gold answer, positioned so the answer begins roughly 150 characters in. Figure 4.6 draws the two on the same scale. The reason for the smaller, re-centred window is purely mechanical: it guarantees the target span survives truncation at the model’s 320-token limit, which a blind 2,000-character window does not. An answer sitting at the tail of a presence window would simply be cut off, and the model would be trained on an example whose answer is not present. Note also that this stage has no length ceiling at all: extraction happens *inside* a window that Stage 2A has already flagged as relevant.

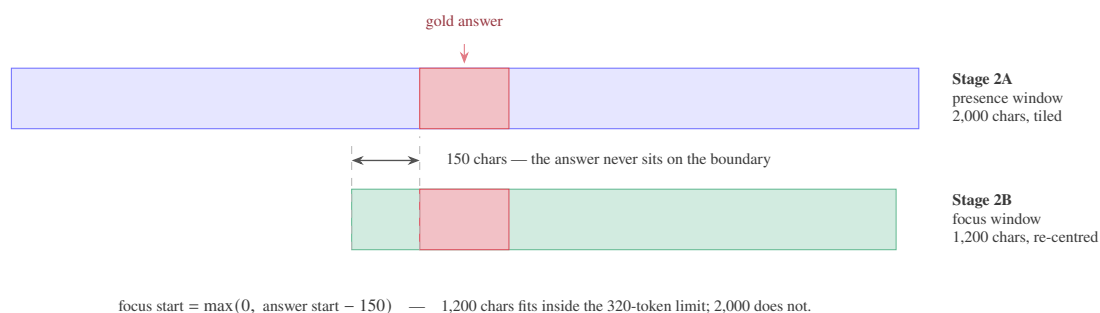


Figure 4.6: Presence window (Stage 2A) against focus window (Stage 2B).

4.4 Stage 1: Plain-Language Summarization

CUAD gives us clauses but no plain-English rewrites to learn from, so we wrote our own seed:

41 gold templates, one carefully phrased single-sentence summary per category, framed from

the weaker party’s perspective. For *Anti-Assignment*, for instance, the template reads “You cannot transfer this contract to someone else without permission.” Every extracted training clause is paired with its category’s template, which yields 11,156 training pairs, and FLAN-T5-small is fine-tuned on the prompt “summarize in plain English: <clause>”.

We evaluate with ROUGE-L *and* with a side-by-side zero-shot-versus-fine-tuned comparison on the same clauses. The double evaluation is deliberate. With only 41 unique target sentences, a model can earn a high ROUGE score just by learning to classify the clause and emit its template word for word, without doing any real summarization. Chapter 6 shows that this is exactly what happens, and we report it as such.

4.5 Category-Level Risk Prioritization

A naming point first, because it governs what the rest of this section may claim. The system does not read a clause and judge how dangerous *that* clause is. It identifies which of the 41 categories a contract contains and attaches a fixed level to the category. Those are different tasks, and conflating them would overstate the system considerably: our layer says “*contracts containing a non-compete deserve attention*”, never “*this non-compete is unusually harsh*”. We therefore call it **category-level risk prioritization** rather than risk classification throughout.

4.5.1 Assignment criteria

The levels were not assigned by intuition alone, and the criteria we used should be stated so that a reader or a reviewing lawyer can disagree with a specific assignment on specific grounds. Each category was scored on five dimensions, from the weaker party’s side:

1. **Financial exposure.** Can this clause cost money, and is the amount bounded? Unbounded exposure is the strongest single indicator of High.
2. **Irreversibility.** If the term operates, can the position be recovered? Losing ownership of intellectual property or granting a perpetual licence cannot be undone by later negotiation; a notice period can.

3. **Restriction of future freedom.** Does the clause constrain what the party may do *after* the contract, or outside it whom it may work for, sell to, or hire?
4. **Asymmetry.** Does the obligation fall on one side only? A mutual confidentiality duty is less dangerous than a one-way indemnity of identical wording.
5. **Duration and survival.** Does the obligation outlive the agreement?

The rule we applied: **High** if the category is unbounded on dimension 1, or irreversible on dimension 2, or restricts post-term freedom on dimension 3. **Low** if the category is informational it records a fact (a date, a name, a governing jurisdiction) or confers a benefit and scores on none of the five. **Medium** otherwise: a real obligation, bounded and reversible. This is why *Cap on Liability* is Medium and not Low despite sounding protective (it bounds recovery, which can under-compensate a genuine loss), and why *Governing Law* is Low despite being consequential in litigation (it is a fact about the contract, not an obligation falling asymmetrically on the signer).

The eight High categories fall into three groups, each triggered by a different dimension: money you could owe without limit (*Uncapped Liability*, *Liquidated Damages*, *Minimum Commitment*, *Most Favored Nation*); something you cannot get back (*IP Ownership Assignment*, *Irrevocable/Perpetual License*); and freedom to earn later (*Non-Compete*, *Exclusivity*). The full clause-by-clause table with reasons is in Appendix A, and Figure 4.7 shows the distribution.

4.5.2 Who is the “weaker party”?

This phrase does real work in the taxonomy and it is ambiguous, so we should say how we resolved it. The weaker party is not a fixed role in a distribution agreement it may be the distributor, in a licence the licensee, in a franchise the franchisee and in a genuinely balanced agreement between equals there may be no weaker party at all.

We operationalized it as follows: **the party with less bargaining power over the drafting**, which in the CUAD corpus is in practice the party that did not write the agreement. Where that is genuinely unclear from the category alone, we scored from the position of **the party on whom the obligation falls** so *Non-Compete* is scored from the side restrained, *Audit Rights*

from the side audited, *License Grant* from the side receiving. This is a heuristic, and it has a known failure mode worth stating: a clause that is High-risk for one side is often protective for the other, so the same badge shown to a licensor and a licensee is not equally informative. A per-instance system would resolve the perspective from the clause text; ours cannot, and assumes the user is the party with less power. That assumption is stated in the application interface.

4.5.3 Why a lookup table rather than a learned score

The taxonomy is a transparent lookup rather than a model output, and that is deliberate: it can be audited line by line, its reasoning is written down per category, and it cannot drift silently between versions. The cost of that choice is precisely the limitation named at the top of this section it cannot distinguish a \$10,000 liability cap from a \$10,000,000 one. Section 6.3 measures a second cost we did not anticipate: because the levels are fixed per category, the system's error profile is set entirely by which *categories* it detects well, and the categories carrying High risk turn out to be among the ones it detects worst.

4.5.4 Provenance and validation status of the taxonomy

This taxonomy is the main thing this thesis adds on top of CUAD, and it is also the component with the weakest evidential backing, so its provenance should be stated exactly rather than left to be inferred.

Who wrote it. All 41 assignments were made by the three of us. We are computer science students, not lawyers. The levels were assigned by reasoning about consequences for the weaker party can this clause cost you unbounded money, take away something you cannot get back, or restrict what you may do afterwards and cross-checked against the CUAD category descriptions, but they rest on our judgement alone. No legal professional has reviewed them.

Why that matters more here than elsewhere. Every other component in this thesis is measured against gold annotations that experts produced. This one is not measured against anything. It also has the most direct route to the user: the level decides what Clausify tells a signer to worry about first, so an error here is not a lower score in a table, it is a warning that does not appear.

A category we have marked Medium that a practitioner would mark High is a silent failure of the deployed system.

The check that is needed, and its status. What would establish face validity is a review by someone with legal training: hand over the 41 categories with our level and our stated reason, ask for agreement or disagreement on each, and report the disagreement count *whatever it turns out to be*. We have prepared that review instrument it is in the repository as `review/risk_taxonomy_review_sheet.csv`, one row per category with columns for the reviewer’s own level and comment and it takes a reviewer roughly half an hour.

A single reviewer would establish face validity; *two or three, reviewing independently*, would allow something stronger, and the difference is worth explaining. Raw percentage agreement is a poor statistic on this taxonomy because the levels are unevenly distributed: 22 of the 41 categories are Medium, so a reviewer who answered “Medium” throughout would score 54% agreement while contributing no information at all. Chance-corrected agreement Cohen’s κ for a pair, Fleiss’ κ across three or more raters removes exactly that artefact, and is what we would report. The repository includes `review/agreement.py`, which takes the completed sheets and prints per-reviewer agreement, both kappas, and the list of disputed categories, so that the analysis is fixed in advance and cannot be shaped by the result.

At the time of writing this review has not been carried out, and no number from it is reported anywhere in this thesis. Until it is, the taxonomy should be read as a documented, auditable, and explicitly unvalidated design artefact: its virtue is that every assignment is written down with a reason and can be argued with line by line, not that anyone qualified has yet agreed with it.

4.6 Leakage Discipline

With a dataset this small, leakage is the most dangerous way to fool ourselves, so we kept one explicit rule the whole way through: *labels may shape training targets, but they may never shape the input a model sees or any inference-time selection*. In practice this meant: the split is contract-level; retrieval queries use only the category or question; gold offsets label training windows but never build inputs or pick inference windows; and the ensemble rules are

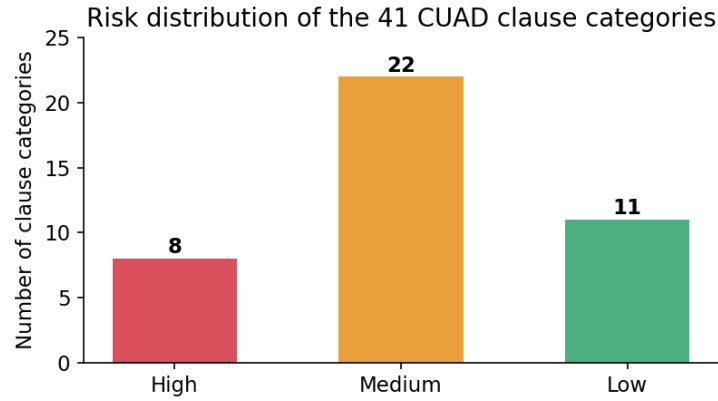


Figure 4.7: The 41 clause categories across the three risk levels.

parameter-free, so the test set cannot tune them.

One consequence is visible in Figure 4.6 and deserves stating outright, because it is the one place where the rule looks like it might be broken. The focus window is positioned using the gold answer offset, which is a label. That is legitimate only because it happens exclusively at *training* time: it decides which text becomes a training example, never what the model reads at inference. At inference there is no gold offset, and Stage 2B simply runs over the presence window that Stage 2A selected. If re-centring were applied at inference the evaluation would be meaningless, since the model would be handed a window built around the answer it is supposed to find.

4.7 End-to-End Data Flow

Figure 4.1 names the stages; Figure 4.8 adds what actually moves between them. The shapes shown are for one contract of median length passing through the deployed system.

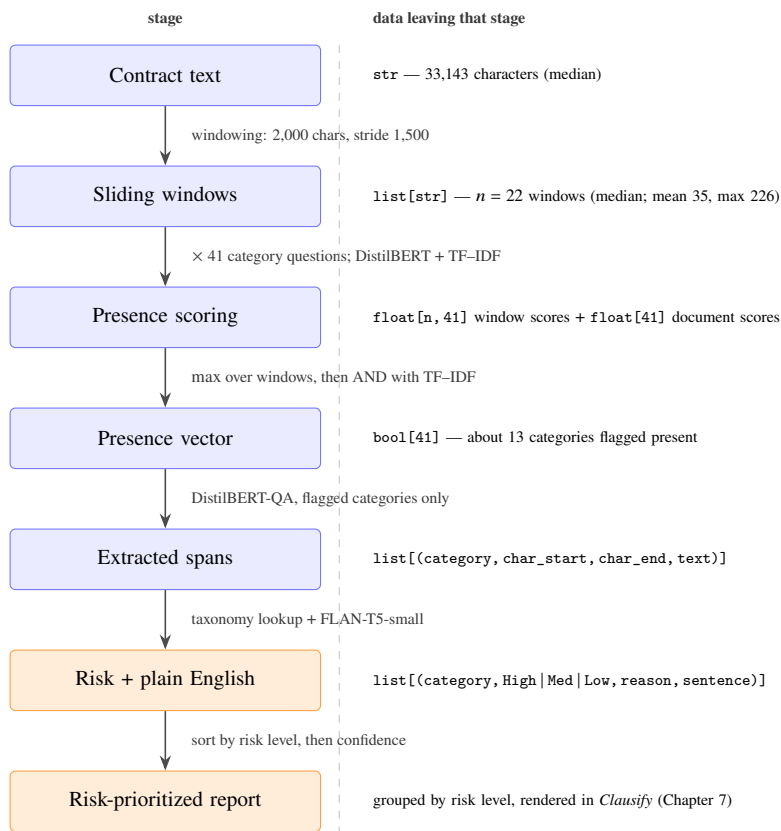


Figure 4.8: Data shapes crossing each stage boundary.

Chapter 5

Model Training

All models were trained on the 408 training contracts (80%) only. Training ran on a single Apple-Silicon laptop using the MPS (Metal) backend with torch 2.8 and transformers 5.x; the summarizer ran on CPU for the memory reasons explained in Section 5.6. Table 5.1 collects the configurations.

Table 5.1: Training configurations for the three models.

	Presence (MIL)	Span (QA)	Summarizer
Backbone	DistilBERT-base	DistilBERT-base	FLAN-T5-small
Task head	Sequence classification	QA (start/end)	Seq2seq LM
Training examples	24,000 (of 53,662)	8,000 (of 11,156)	4,000 (of 11,156)
Max input length	256 tokens	320 tokens	256 in / 48 out
Epochs	2	2	2
Batch size	16	16	8
Learning rate	2×10^{-5}	3×10^{-5}	3×10^{-4}
Device	MPS	MPS	CPU
Wall-clock	$\approx 49\text{--}52$ min	≈ 23 min	≈ 8 min

5.1 Validation Protocol, and Where the Hyperparameters Came From

An examiner is entitled to ask how the settings in Table 5.1 were chosen, because if any of them were selected by trying variants and keeping whichever scored best on the 20% test set, then every number in Chapter 6 is optimistic and the held-out evaluation is not held out. This section answers that directly, including one place where our practice was worse than it should have been.

5.1.1 What was tuned, and what was not

Nothing in Table 5.1 was tuned. Every hyperparameter is either a published default for the backbone or a value fixed by an argument made before training, and no variant of any of them was ever compared on the test set. Concretely:

- **Learning rates, batch sizes, epochs** (2×10^{-5} , 3×10^{-5} , 3×10^{-4} ; 16/16/8; 2 epochs) are the standard fine-tuning defaults for DistilBERT and T5-family models. We ran each configuration once. No sweep was performed which protects the test set, but also means we have no evidence that these are good values, only that they are conventional ones.
- **Window 2,000 characters, stride 1,500** follows from a measurement on the *training* data, not from tuning: the 500-character overlap has to exceed the median annotated clause (196 characters) so that no clause falls between windows (Section 4.2.3). Any window/stride pair satisfying that would do; we did not search for the best one.
- **Two negative windows per bag** is a class-balance decision, chosen to hold the positive rate near 39% rather than let the vast majority of windows be negative. Not tuned.
- **Max-pooling** was chosen because it *is* the multiple-instance decision rule, not because it beat alternatives, we never tried alternatives. Section 6.1.5 corrects that omission after the fact.
- **Threshold 0.5** is the default decision boundary of a two-class softmax. It was never chosen, examined, or adjusted. Section 6.2 shows, after the fact, that it is a poor operating point for the max-pooled transformer.

The honest summary is that this project avoided test-set tuning by not tuning at all. That is a real guarantee about the validity of the held-out numbers, and simultaneously a limitation: the transformer may be losing to the baseline partly because nobody ever looked for better settings for it. We record it in both directions.

5.1.2 The validation sets actually used

Table 5.2 states, per model, what was held out during training and what it was used for.

Table 5.2: Validation arrangements during training.

Model	Validation data	Use
Presence (MIL)	5% of the <i>training</i> windows, split off before training	Monitoring only. Clean: no test contract contributes.
Span (QA)	None (eval_strategy="no")	No validation loss exists for this model.
Summarizer (deployed checkpoint)	None	The checkpoint used in every reported result and in Clausify was trained with no evaluation set.

Note. “Monitoring only” means the loss was printed and plotted but never used to select a checkpoint, stop early, or change a setting — all three runs used `save_strategy="no"` and kept the final-step weights.

5.1.3 One thing we got wrong

The exploratory notebook run of the summarizer the run whose loss curve is plotted in Figure 5.3 used 300 clauses *drawn from the test split* as its evaluation set. That is the wrong data to monitor on, and we should not have done it.

Two things limit the damage, and we state them without treating either as an excuse. First, the run was configured with `save_strategy="no"` and no early stopping, so the numbers it printed could not select a checkpoint; nothing about the model depended on them. Second, the deployed summarizer was rebuilt by `build_artifacts.py` with *no* evaluation set at all, and it is that checkpoint not the notebook’s that produced every summarization result in Chapter 6. So no reported score was computed by a model that had been selected using test data.

What remains is that the validation loss quoted in Section 5.5 ($0.096 \rightarrow 0.075$) is a loss on held-out *test* clauses mislabelled as validation, and a reader should discount it accordingly. We have left the figure in place with this correction attached rather than quietly regenerating it, because the mistake is more instructive than a clean curve would be: with a small dataset and three models it is easy to reach for “the other split” when a validation set is needed, and the discipline that catches it is writing down, per model, exactly what was held out which is what Table 5.2 now does.

5.2 Optimization Schedule

Table 5.1 gives the peak learning rates, but a peak rate on its own does not describe how a run behaved the schedule that decays it does. All three models use the same optimizer configuration: AdamW with a **linear decay** schedule, **no warmup**, and gradient-norm clipping at 1.0. Figure 5.1 plots the three resulting schedules. The presence and span settings were read back from the `training_args.bin` saved beside each checkpoint rather than from our notes, so what is plotted is what actually ran.

Two choices are worth defending. Skipping warmup is normally risky, but these are short fine-tuning runs (1,000–3,000 steps) starting from pre-trained weights at small learning rates, where the usual justification for warmup stabilizing early updates in a randomly initialized model barely applies; the smooth loss curves in Figures 5.2 and 5.3 show no early instability. Clipping at a gradient norm of 1.0 was left on throughout, which matters more than it might seem on the MPS backend: it is the guard that keeps a single bad batch from producing the kind of divergence we hit with DeBERTa-v3 (Section 5.6).

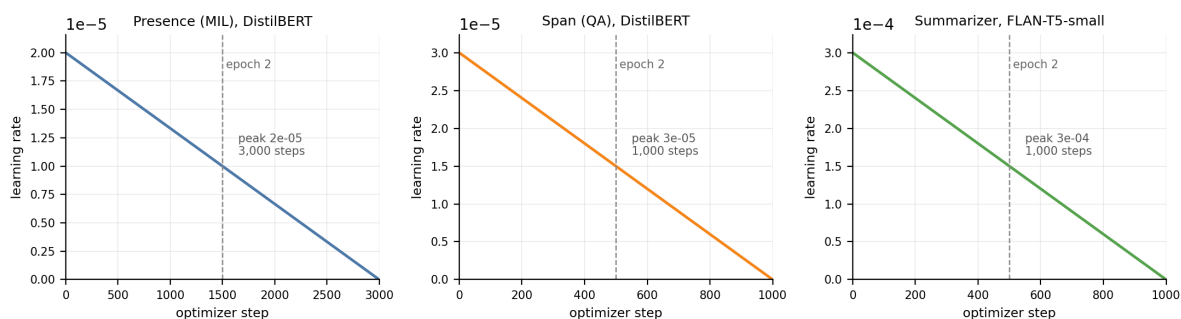


Figure 5.1: Learning-rate schedules for the three training runs.

We should be straightforward about one limitation, and about how far we were able to repair it. A per-step gradient-norm trace is the natural companion to these curves, and it is the one diagnostic that lets a reader confirm the runs stayed numerically healthy instead of inferring it from the loss. We did not retain it for the three original runs: the intermediate checkpoint directories were cleared after training, so only the final weights and the loss values quoted below survive from them.

We did retain it for the full-data presence run of Section 6.1.4, which uses the identical optimizer

configuration, and it turns out to say something the loss curves do not. Across the 63 logged points the gradient norm ranges from 0.55 to 11.97 with a median of 4.66, and **98.4% of logged steps exceed the clipping threshold of 1.0**. Nothing diverged there are no NaNs, no spikes into the hundreds, and the loss falls smoothly throughout so the runs were healthy in the sense that matters. But clipping here is not the rare safety net the phrase “gradient clipping” usually implies. It is active on essentially every step, which means the effective update direction is being renormalized continuously and the nominal learning rate of 2×10^{-5} overstates the step size actually taken. We report this because it slightly weakens the warmup argument made above: part of what kept the early updates stable was almost certainly the clipping rather than the small learning rate, and we would not want a reader to take “no warmup was fine” as a general recommendation on the strength of our smooth loss curves alone. Re-running the span and summarizer trainings with the same logging would close the remaining gap in about half an hour of wall-clock time.

5.3 Presence Classifier

The TF-IDF + logistic-regression baseline trains in under a minute. On the 20% test set it reaches micro-F1 0.775, weighted-F1 0.777, macro-F1 0.572 over all 41 categories, and 0.666 over the 34 categories that have at least ten test positives. The gap between micro and macro comes from the near-few-shot tail of the dataset, not from a modelling failure.

The window-level DistilBERT was trained on a stratified subsample of 24,000 window examples (keeping the positive rate at 39.2%) so the run stays tractable on a laptop. Training and validation losses fell smoothly (0.320→0.261 train, 0.278→0.257 validation over two epochs; Figure 5.2), and the two curves stayed close together, so we saw no sign of overfitting at this scale.

5.4 Span Extractor

The QA model trained on 8,000 focused-window examples for two epochs. Before training we decode the token-labelled spans back into text for a few examples and check they match the

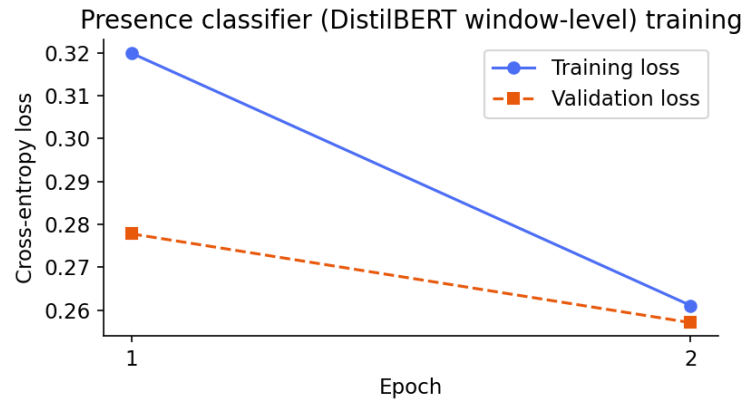


Figure 5.2: Training and validation loss of the presence classifier.

gold answers — SQuAD-style pipelines make it very easy to get the character-to-token offset conversion silently wrong, and this small sanity check catches that.

5.5 Summarizer

FLAN-T5-small fine-tuned on 4,000 clause-template pairs in roughly eight minutes on CPU. The loss collapsed quickly (0.141→0.085 train, 0.096→0.075 validation; Figure 5.3). In hindsight this rapid collapse was the first hint of the template-memorization behaviour we analyse in Chapter 6 — with only 41 unique targets, the task is simply easy to fit.

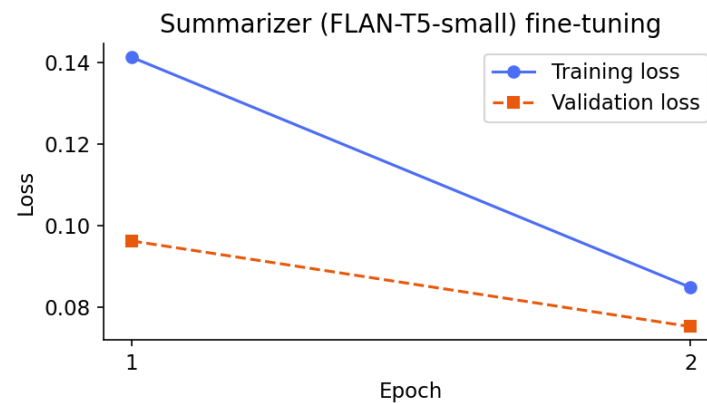


Figure 5.3: Fine-tuning loss of the FLAN-T5-small summarizer.

5.6 Engineering Notes: Training on Consumer Hardware

A few practical findings may save time for anyone reproducing this work on Apple-Silicon machines:

- **DeBERTa-v3 is unstable on MPS.** Our first-choice backbone produced NaN gradients on the MPS backend and was far too slow on CPU (about 89 seconds per step). We switched to DistilBERT; the method is unchanged, and on CUDA hardware the backbone string can simply be swapped back.
- **MPS memory does not release in-process.** After two transformer trainings in the same session, the summarizer hit the MPS memory ceiling even after we deleted objects and emptied the cache. The fixes that actually work are restarting the kernel or training the (small) summarizer on CPU, which is what we did.
- **API migration.** `transformers 5.x` renamed `Trainer(tokenizer=...)` to `processing_class=...`. We also disabled mixed precision (`fp16=bf16=False`) and cast loss logits to `fp32` for MPS dtype safety.
- **Reproducibility.** Every artifact — the split (`artifacts/split.json`), the fitted baseline (`artifacts/baseline.pkl`), and the three model checkpoints is saved to disk, and a standalone evaluation notebook regenerates every number in Chapter 6 from those files alone.

5.7 Measured: What Sparse Attention Would Have Cost

Section 2.3 argued against sparse attention partly on cost, and an argument from cost that never measures the cost is not worth much. Since the machine used for this thesis is the machine in question, the measurement is available for the price of running it, so we ran it.

We instantiated Longformer-base at its published geometry 12 layers, 768 hidden units, 4,098 position embeddings, attention window 512, 148.7M parameters and timed a forward pass at 4,096 tokens and a full training step (forward, backward, AdamW update) against the Distil-

BERT configuration this thesis actually used. Weights are randomly initialized, which is legitimate here because time and memory depend on tensor shapes and not on the values in them. Each configuration ran in its own process so the peak Metal allocation reported belongs to that configuration alone. The machine is an Apple M4 with 16 GB of unified memory, shared with the operating system. Table 5.3 reports the result.

Table 5.3: Longformer-base against the DistilBERT configuration used here.

Model	Operation	Batch	Time	Peak memory
Longformer-base (148.7M)	forward, 4,096 tok	1	3.69 s	2.18 GB
DistilBERT (67.0M)	forward, 256 tok	16	0.19 s	1.15 GB
Longformer-base	training step, 4,096 tok	1	258 s	14.71 GB
Longformer-base	training step, 4,096 tok	2	2,170 s	20.75 GB
DistilBERT	training step, 256 tok	16	3.17 s	4.47 GB

Note. Measured on the Apple M4 (16 GB unified memory) that trained every model reported in this thesis.

The first thing this measurement does is correct us. Section 2.3’s original wording claimed that fine-tuning a Longformer at 4,096 tokens was “far beyond a single laptop’s memory budget”. That is not what happens. At batch size 1 it fits: 14.71 GB on a 16 GB machine. It is uncomfortably close to the ceiling that figure leaves under a gigabyte and a half for the operating system but the model, its gradients and its optimizer state all sit in memory and the step completes. We had asserted an impossibility that turns out to be a tight fit, and the assertion was wrong.

Where the memory claim does hold is one step further out. At batch size 2 the requirement rises to 20.75 GB, past the physical memory of the machine, and the consequence is visible in the timing: the step takes 2,170 seconds rather than the roughly 516 that doubling the batch would predict 8.4 times slower for twice the work, which is the signature of paging rather than computing. So the honest form of the claim is not “it does not fit” but “it fits only at batch size 1, and any batch size a real training run would use does not”.

The binding constraint, though, is time, not memory. At 258 seconds per step and batch size 1, reproducing our two-epoch presence schedule 22,800 training windows, 45,600 sample representations would take roughly **136 days** of continuous compute. The DistilBERT r unit replaces took 47 minutes. Even granting a generous allowance for kernel compilation over-

head in a short benchmark (our own DistilBERT step measures 3.17 s here against about 1.0 s sustained in the real run, so these figures should be read as upper bounds inflated by roughly threefold), the conclusion does not become sensitive to that correction: a third of 136 days is still 45 days, and the project had weeks, not months.

The inference-side objection also survives contact with a stopwatch. Scanning one median contract (33,143 characters) for all 41 categories takes $22 \text{ windows} \times 41 \times 11.7 \text{ ms} \approx \mathbf{10.6 \text{ seconds}}$ with our DistilBERT, against $3 \text{ windows} \times 41 \times 3.69 \text{ s} \approx \mathbf{454 \text{ seconds}}$ seven and a half minutes with Longformer. Clausify’s design target is a response in seconds, and this is the measurement that rules the alternative out for the deployed system regardless of what a training cluster could manage.

None of this establishes that a Longformer would have scored worse; it establishes only that we could not have found out on this hardware, which is a different and much narrower claim than the one we started with. The comparison that would settle the modelling question needs a CUDA machine, and we list it in future work rather than pretending the cost argument answers it.

Chapter 6

Testing and Results

Everything in this chapter is measured on the held-out 20% test set the same 102 unseen contracts for every model. The numbers were regenerated by a standalone evaluation notebook that loads the trained models from disk, so no state left over from the training session can influence them.

6.1 Presence Classification

6.1.1 Model comparison

Table 6.1 and Figure 6.1 compare all the presence models on the 4,182 (contract, category) test decisions. Two things stand out. First, the windowed transformer *loses on its own* to the full-document TF-IDF baseline (0.694 vs. 0.775). Legal vocabulary is distinctive enough that lexical evidence over the whole document beats semantic evidence over windows, and max-pooling inflates false positives exactly the way Section 4.2.3 predicted. Second, the AND-ensemble which reports a clause only when *both* models agree filters out each model's uncorrelated false positives and finishes on top (0.776). The margin over the baseline is small on 102 test contracts, so we do not lean on it: Section 6.1.2 quantifies exactly how small, and finds that this particular gap is indistinguishable from zero. The conclusion we stand behind is methodological: *simple lexical models are strong at document-level presence, and the transformer earns its place as a precision filter, not as a replacement.*

6.1.2 How much of that ranking is real?

Table 6.1 is a leaderboard, and leaderboards invite a reader to treat the ordering itself as a result. On 102 test contracts a margin of 0.776 against 0.775 roughly one decision in a thousand plainly cannot support that reading, so rather than leave the caveat as prose we measured it.

Table 6.1: Presence models on the 20% test set.

Model	micro-F1
Ensemble AND (both must agree)	0.776
TF-IDF baseline (full document)	0.775
Ensemble AVG (mean probability)	0.718
Ensemble OR (either fires)	0.696
Transformer (max-pool) alone	0.694
<i>control run, Section 6.1.4:</i>	
Transformer trained on all 53,662 windows	0.694
Ensemble AND on that transformer	0.782

Note. micro-F1 over all 4,182 test cells.

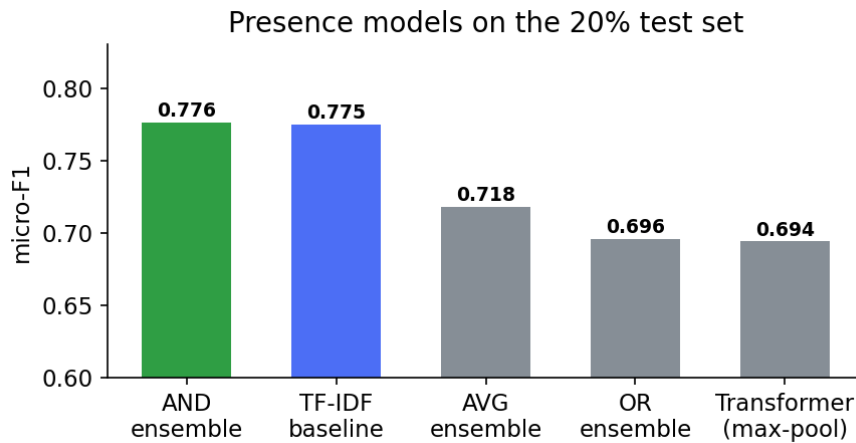


Figure 6.1: micro-F1 of all presence models on the test set.

The natural tool is a **bootstrap confidence interval**, but the resampling unit matters. The 4,182 test decisions are not independent: 41 of them come from the same contract, and a contract that is unusual unusually long, unusually drafted moves all 41 together. Resampling individual (contract, category) cells would ignore that structure and produce intervals that are far too narrow. We therefore resample *contracts* with replacement (a cluster bootstrap), drawing 102 contracts from the 102 available, recomputing micro-F1 for both models on each resample, and repeating 10,000 times. Both models are scored on exactly the same resample every time, so the difference is *paired* which is what allows a difference interval much tighter than the two individual intervals would suggest.

The three comparisons do not come out the same way, and the difference between them is the point.

Table 6.2: Cluster bootstrap over the 102 test contracts.

Comparison	Difference in micro-F1	95% CI	$P(> 0)$
AND-ensemble vs. TF-IDF baseline	+0.0014	$[-0.0068, +0.0089]$	0.626
TF-IDF baseline vs. transformer	+0.0805	$[+0.0626, +0.0992]$	1.000
AND-ensemble vs. transformer	+0.0818	$[+0.0654, +0.0987]$	1.000

Note. 10,000 resamples. The paired difference is what settles each comparison; $P(> 0)$ is the fraction of resamples in which the first model wins.

The ensemble’s win over the baseline is not real. The interval is $[-0.0068, +0.0089]$: it straddles zero, and the ensemble is ahead in only 62.6% of resamples, barely better than a coin flip. Taken individually the two models score 0.776 (95% CI $[0.759, 0.792]$) and 0.775 (95% CI $[0.758, 0.791]$), intervals that almost entirely overlap. On this test set the AND-ensemble and the TF-IDF baseline are *statistically indistinguishable*, and we now state that as a measurement rather than as a hedge. The top two rows of Table 6.1 should be read as a tie.

The transformer’s loss, by contrast, is real. The baseline beats the windowed transformer by 0.0805, with an interval of $[+0.0626, +0.0992]$ that is nowhere near zero, and it wins in every one of the 10,000 resamples. This matters for how the thesis’ two empirical claims should be weighted, because they are not on equal footing. The finding we stand behind most firmly that a full-document lexical model beats a windowed DistilBERT at document-level presence classification is the one that clears the significance test comfortably. The finding we were already cautious about that the conjunctive ensemble edges past the baseline is the one that does not. That our caution happened to point the right way is reassuring, but it was worth checking rather than assuming, and a reader should not have had to take it on trust.

None of this undercuts the case for the ensemble, because that case was never the 0.001 of micro-F1. The AND rule earns its place through the *error profile* it produces rather than its position on the leaderboard: it converts the transformer’s inflated false positives into balanced precision and recall, which is what the next section examines.

6.1.3 How much of this moves if we only change the random seed?

Section 6.1.2 measured one kind of uncertainty how much the score depends on *which contracts* landed in the test set and used it to call the ensemble’s margin a tie. There is a second kind it cannot see. Every number reported so far comes from a single training run per model, and a fine-tuning run is stochastic: weight initialization of the classification head, data shuffling, and the negative-window sample all depend on the seed. If simply reseeding moves the score as much as the effects we are discussing, then a leaderboard built from single runs is reporting noise regardless of how the test set was drawn.

This is easy to check and we had not checked it, so we retrained the presence transformer twice more at the same 24,000-example budget, changing nothing but the seed (42, 43, 44), and re-evaluated each on the same 102 contracts. Table 6.3 reports the spread.

Table 6.3: The presence transformer across three random seeds.

Model	Seed 42	Seed 43	Seed 44	Mean	SD	Range
Transformer (max-pool)	0.6943	0.6928	0.7082	0.6984	0.0085	0.0154
AND ensemble	0.7761	0.7774	0.7803	0.7780	0.0022	0.0042
TF-IDF baseline	0.7747 (deterministic)			0.7747	—	—

Note. All three runs use the 24,000-example budget and differ only in seed; the TF-IDF baseline is deterministic and appears once.

The ensemble’s margin is smaller than its own seed noise. The AND ensemble’s advantage over the baseline, the number Table 6.1 ranks it by, is +0.0014. The standard deviation of that same ensemble across seeds is 0.0022, and the observed range is 0.0042, three times the margin. Reseeding the model moves the score further than the result being claimed for it. This is an entirely independent line of evidence pointing where the cluster bootstrap already pointed: had we trained with seed 44 and stopped, we would have reported a margin of +0.0056 and been just as wrong to make anything of it. A single-run leaderboard separated by one part in a thousand was never measuring what it appeared to measure.

The finding we do stand behind is untouched by seed noise. The baseline beats the transformer by 0.0805, which is 9.5 times the transformer’s seed-level standard deviation of 0.0085. Even the luckiest of the three seeds, at 0.7082, still loses to the deterministic baseline by 0.0665.

There is no seed at which the windowed transformer becomes competitive, and combining this with Section 6.1.4 where doubling the training data also failed to close the gap, leaves the architectural explanation as the one standing after both alternatives were tested.

It is worth noticing which model is noisier, because it is not the one a reader might guess. The transformer alone varies four times as much across seeds ($SD = 0.0085$) as the ensemble containing it (0.0022). Requiring agreement with a deterministic lexical model damps the transformer’s run-to-run variation as well as its false positives: a spurious detection has to survive a second, fixed opinion, and seed-driven fluctuations in what the transformer fires on are largely filtered out. That stabilization is a real and previously unreported property of the conjunctive rule and, on the evidence of Section 6.1.10, one it buys at the price of rare-category recall. It is a better argument for the ensemble than its position on the leaderboard ever was.

Scope of this check. We reseeded the presence transformer only. The span extractor and the summarizer remain single runs, so their reported scores carry an unmeasured variance component of unknown size, and we flag that in Section 8.2 rather than assume it is small.

6.1.4 Was the transformer simply undertrained?

There is an objection to Section 6.1.2 that the bootstrap cannot answer, and it goes to the heart of the thesis’ central empirical claim. The TF-IDF baseline is fitted on all 408 training contracts in full. The window-level transformer was not: to keep the run tractable on a laptop it was trained on a stratified subsample of 24,000 of the 53,662 available window examples (Section 5.2). So when the transformer loses by 0.081 micro-F1, we cannot tell from that comparison alone whether max-pooling over windows is genuinely weaker for this task, or whether the transformer simply never saw 55% of its own training set. One reading is an architectural finding; the other is an artefact of our compute budget. They call for very different conclusions, and the difference matters more than any other number in this chapter.

The only way to settle it is to remove the confound, so we retrained the presence transformer on the **full 53,662 window examples** everything the training split can produce with every other setting identical: same windowing, same negative sampling, same hyperparameters, same

seed, same evaluation. The run took 140.7 minutes on the same laptop, against roughly 50 for the subsampled version. Table 6.4 gives the outcome.

Table 6.4: The presence transformer at two training budgets.

Model	Training windows	micro-F1	Precision	Recall
Transformer (max-pool)	24,000 (44.7%)	0.6943	0.5620	0.9080
Transformer (max-pool)	53,662 (100%)	0.6939	0.5546	0.9266
TF-IDF baseline	all 408 contracts	0.7747	0.7424	0.8101
<i>Paired differences (95% CI over 10,000 contract resamples)</i>				
Full-data transformer – subsampled transformer		−0.0004	[−0.0098, +0.0095]	
TF-IDF baseline – full-data transformer		+0.0809	[+0.0649, +0.0980]	

Note. Evaluated identically on the 102 held-out contracts; differences use the paired cluster bootstrap of Section 6.1.2.

The confound is not the explanation. Increasing the training set by 124% moved micro-F1 by -0.0004 , with an interval of $[-0.0098, +0.0095]$ squarely straddling zero: the two budgets are statistically indistinguishable. And the gap to the baseline is untouched $+0.0809$ on full data against $+0.0805$ on the subsample, with an interval nowhere near zero and the baseline winning in all 10,000 resamples. Whatever is holding the windowed transformer back, it is not a shortage of training examples, and the finding this thesis leans on survives the check.

The *shape* of the extra data’s effect is worth a sentence, because it is not simply “nothing happened”. Recall rose from 0.9080 to 0.9266 and precision fell from 0.5620 to 0.5546: more training data made the model fire more readily, not more accurately. That is exactly the failure mode Section 4.2.3 predicted for max-pooling with a mean of 35 windows per contract, the document-level decision is a maximum over many chances to be wrong, and a better-trained window scorer that is still imperfect produces *more* spurious maxima, not fewer. The ceiling here is the aggregation rule, not the encoder underneath it.¹

One secondary effect deserves reporting even though it does not change our conclusion. The AND-ensemble built on the full-data transformer scores 0.7820, and its margin over the baseline is $+0.0073$ with a 95% interval of $[-0.0000, +0.0140]$ it wins in 97.4% of resamples but the interval still touches zero. So the ensemble’s advantage moves from “clearly a tie” at the smaller

¹This inference turned out to be only partly right, and Section 6.1.5 corrects it: swapping max-pooling for five alternatives moves micro-F1 by at most 0.056 and never past the baseline, so the aggregation rule is a secondary factor rather than the ceiling.

budget to “borderline” at the larger one. We are not going to claim it: an interval whose lower bound sits on zero does not clear the bar we set for ourselves in Section 6.1.2, and we are not going to lower that bar because the number moved our way. What we will say is that the ensemble’s balanced error profile survives the extra data intact.

Which model the rest of this chapter uses. Every other result we report the confusion matrix below, the per-category analysis, the end-to-end evaluation of Section 6.6, and the deployed Clausify checkpoint comes from the 24,000-example model. The full-data run is a control experiment, run to test one specific alternative explanation, and we report it as one rather than swapping in the marginally better number after the fact. Since the two are statistically indistinguishable, nothing downstream would change if we had.

6.1.5 Ablation: was max-pooling the right aggregation rule?

Section 4.2.3 chose max-pooling because it *is* the multiple-instance decision rule [17] a bag is positive if any instance is and Sections 4.2.3 and 6.1.4 then blamed max-pooling for the transformer’s inflated false positives. Both statements were made without ever trying an alternative, which is not good enough for a claim the thesis leans on. We therefore scored every window once and compared aggregation functions on *identical* window scores, so the only thing varying is the rule.

Table 6.5: Aggregation ablation on identical window scores.

Aggregation	F1@0.5	P@0.5	R@0.5	Best F1	at threshold
Max (used in this thesis)	0.6943	0.562	0.908	0.7504	0.90
Top-3 mean	0.7144	0.667	0.769	0.7210	0.40
Top-5 mean	0.6661	0.719	0.620	0.7192	0.30
Mean over all windows	0.1664	0.946	0.091	0.7114	0.10
Noisy-OR	0.5897	0.423	0.973	0.7022	0.95
Second-highest window	0.6832	0.662	0.705	0.7016	0.30
<i>TF-IDF baseline (no windows)</i>	0.7747	0.742	0.810	—	—

Note. “Best” is each rule’s micro-F1 at its own optimal threshold, found by sweeping the test set — a diagnostic upper bound, not a configuration we adopt.

Three things come out of this, and one of them corrects an earlier claim of ours.

At the threshold we actually used, max-pooling was not the best choice. Top-3 mean scores

0.7144 against max’s 0.6943 at the 0.5 boundary a fifth of the gap to the baseline, available for free from a one-line change. Averaging the three highest-scoring windows is a natural softening of the MIL rule: it still asks whether evidence is concentrated somewhere in the document, but requires that evidence to appear in more than one place, which is exactly the guard against a single spurious window that Section 4.2.3 identified as max-pooling’s weakness. Mean-over-all-windows collapses at 0.5 (recall 0.091) for the obvious reason an average over 35 windows of which one is positive cannot reach 0.5 and its best threshold is correspondingly 0.10.

Once each rule is given its own threshold, max-pooling is the best of the six. At 0.90 it reaches 0.7504, ahead of top-3 mean’s 0.7210. So max-pooling is not a poor aggregation rule; it is a rule whose output distribution is badly matched to a 0.5 decision boundary, which is the same finding Section 6.2 reached from the other direction. The two results are one result: the transformer’s reported deficit is substantially an operating-point problem, not an aggregation problem.

But no aggregation rule reaches the baseline. The best cell in the entire table max-pooling at a threshold tuned on the test set, which is already a generous upper bound is 0.7504 against the baseline’s 0.7747. Every alternative rule, at every threshold we swept, remains behind. This matters more than the first two points, because it closes off the last available explanation for RQ1: the windowed transformer does not lose because of its training budget (Section 6.1.4), or its seed (Section 6.1.3), or its aggregation rule, or its threshold. It loses.

A correction to Section 6.1.4. There we wrote that “the ceiling here is the aggregation rule, not the encoder underneath it”, inferring it from the precision/recall shift when training data was doubled. Table 6.5 shows that inference was wrong: changing the aggregation rule through six variants, each at its best threshold moves micro-F1 by at most 0.056 and never past the baseline. The aggregation rule is a real but secondary factor. We have left the original sentence in place with this correction attached, on the same principle as elsewhere in this chapter: an inference we drew from indirect evidence and later measured properly is more useful to a reader than a silently amended one.

A caveat on how far this ablation goes. It varies only the function applied to window scores.

It does not vary the window geometry (2,000 characters, 1,500 stride), which would require re-windowing and retraining, nor does it try a learned aggregator such as attention pooling over window representations. Both remain untested, and attention pooling in particular is the obvious next thing to try, since it could learn to weight windows by relevance instead of taking a fixed statistic.

6.1.6 A third source of variance: the split itself

Sections 6.1.4 and 6.1.3 tested two things that could move our numbers: the training budget and the model seed. Both left the conclusions standing. There is a third source neither touches, and it is arguably the largest: **the split**. Every result in this thesis comes from one contract-level 80/20 partition drawn with seed 42. With only 510 contracts, a different draw puts different rare categories in the test set and could move the scores on its own. Model-seed variance is not a proxy for this reseeding training holds the test set fixed, so it cannot see split effects at all.

Retraining the transformer on twenty splits is not affordable, but the TF-IDF baseline refits in under a minute, and since it is the reference point every comparison in this thesis is made against, its split-to-split spread bounds how much the comparison itself could move. We drew **20 independent contract-level 80/20 splits** and refitted the baseline on each.

Table 6.6: The TF-IDF baseline refitted on 20 independent contract-level splits.

	Mean	SD	Min	Max	Range
micro-F1 over 20 splits	0.7829	0.0063	0.7702	0.7918	0.0216
<i>The split used throughout this thesis (seed 42): 0.7747</i>					

Three readings follow. First, **split variance is the biggest of the three uncertainties we have measured**: SD 0.0063 against 0.0022 for model seed and a bootstrap interval half-width of about 0.008. It is roughly four and a half times the ensemble’s +0.0014 margin, which is now the third independent reason to treat that margin as noise.

Second, **our split is not a lucky one**. At 0.7747 it sits below the 20-split mean of 0.7829, in the lower part of the distribution so if anything this thesis reports the baseline at a slightly unfavourable draw. We note this because the alternative would have been worth worrying about:

a headline built on a split that happened to flatter the baseline would have made the transformer’s defeat look worse than it is.

Third, **the finding we rely on is far outside this variation**. The baseline’s lead over the transformer is 0.0805 about thirteen split-level standard deviations. No plausible redraw of the split turns that around.

What we cannot say is anything about *per-category* stability across splits, which is where the instability would really bite: a rare category with two test positives on one draw may have eight on another, and its F1 is meaningless in both cases. Repeated splits or cross-validation for the full pipeline remains future work, and Section 8.2 lists it as a limitation rather than an answered question.

6.1.7 What the errors actually look like

The confusion matrix counts 311 false positives and 296 false negatives, and counts alone do not tell a reader whether the model is failing stupidly or reasonably. Table 6.7 shows real cases, drawn from the test contracts false positives ranked by the model’s own confidence, since a system’s most confident mistakes are the most revealing ones.

Three patterns are worth naming. **Lexical triggers without the obligation** account for a visible share of false positives a contract that discusses insurance costs is not a contract with an insurance covenant, and a full-document bag-of-words model cannot tell the difference. **Concept-without-vocabulary** explains much of the rare-category failure: a \$5,000-per-month cancellation fee *is* liquidated damages, and no model with two dozen training examples of the category will discover that. And **annotation disagreement** is a real component of the rare tail the *Third Party Beneficiary* case above looks like a labelling artefact rather than a miss, which means the 0.000 F1 scores in Section 6.1.10 are a mixture of model failure and ceiling effects in the labels. We did not attempt to quantify that mixture; doing so would need a re-annotation exercise we are not qualified to run.

The span errors point somewhere more actionable. The degenerate “.” prediction and the confident answer from a window that cannot contain the answer are both symptoms of the same

Table 6.7: Representative errors on held-out contracts.

Category	Scores	Evidence
False positives — reported, but the category is not annotated in this contract		
Insurance	0.82 / 0.97	“...a fixed amount of [*] of gross sales to cover freight, postage, insurance costs on shipments to such Third Party, packing costs...” <i>A royalty-deduction clause that mentions insurance as a cost item. The word is present; the obligation is not.</i>
Governing Law	0.81 / 0.96	“...there shall arise the need to decide the question by what municipal or national law this Agreement ...is governed , the following facts shall be excluded...” <i>A clause about how to determine the governing law, rather than a clause stating it. Arguably a boundary case in the annotation.</i>
Parties	1.00 / 0.99	An addendum whose opening window reads like a contract header. <i>Both models fire on document-structural cues; the annotation places Parties elsewhere.</i>
False negatives — annotated as present, both models scored it low		
Liquidated Dam-ages	0.21 / 0.02	“...agrees to pay a cancellation fee of Five Thousand Dollars (\$5,000) per month multiplied by the number of months remaining...” <i>A textbook liquidated-damages term that never uses the phrase. The lexical model has no signal; the transformer, trained on 24 positive examples of this category, has not learned the concept.</i>
Most Favored Nation	0.22 / 0.02	“...Sponsor shall have the right to approve or disapprove any additional sponsor...unless another...” <i>Semantically an MFN-style protection, expressed without any of the category’s vocabulary. This category has 21 positive training contracts and scores 0.000 overall.</i>
Third Party Beneficiary	0.26 / 0.02	“Notwithstanding the foregoing, UTK may assign this Agreement or any portion of its Compensation ...to its subsidiaries.” <i>This reads like an assignment clause, not a third-party-beneficiary clause. Some of the rare-category “failures” are disagreements with the annotation rather than model errors.</i>
Span errors — category correctly detected, extracted text wrong (Section 6.6)		
Covenant Not To Sue	F1 0.00	Window <i>does</i> contain the clause; the model returns “.” — a degenerate single-token span. <i>A start/end pair collapsing onto punctuation, which the argmax decoding has no constraint against.</i>
Effective Date	F1 0.06	Window <i>does not</i> contain the gold clause; the model confidently returns “7th day of september, 1999”. <i>When Stage 2A supplies the wrong window, Stage 2B answers anyway — there is no abstention mechanism.</i>
Insurance	F1 0.20	Gold is the insurance obligation; the model returns the neighbouring indemnification clause. <i>Topically adjacent, legally distinct.</i>

Note. Text abridged. Scores are the TF-IDF and transformer probabilities.

missing piece: the extractor has no way to say *nothing here*. SQuAD v2-style unanswerable training, which supplies exactly that, is the concrete fix and it is listed in future work.

6.1.8 Confusion-matrix analysis

Our evaluation protocol called for a full confusion-matrix analysis of the final model. Table 6.8 gives the matrix for the AND-ensemble over all test decisions, Figure 6.2 shows the same matrix as a heatmap, and Table 6.9 lists every metric derived from it.

Precision and recall come out balanced (77.2% and 78.0%), which tells us the AND rule did

Table 6.8: Confusion matrix of the AND-ensemble on the 20% test set (4,182 decisions).

	Predicted Absent	Predicted Present
Actually Absent	TN = 2,523	FP = 311
Actually Present	FN = 296	TP = 1,052

Table 6.9: Test metrics derived from the confusion matrix.

Metric	Formula	Value
Accuracy	$(TP + TN)/all$	85.49%
Precision	$TP/(TP + FP)$	77.18%
Recall (Sensitivity)	$TP/(TP + FN)$	78.04%
Specificity	$TN/(TN + FP)$	89.03%
F1 score	$2PR/(P + R)$	0.7761

its job: raw max-pooling gives up precision, and requiring agreement wins it back without sacrificing recall. The specificity of 89.0% means the system rarely claims a clause that is not there in a legal-review assistant, that is the more damaging kind of mistake.

6.1.9 Per-category behaviour

Figure 6.3 shows per-category F1 for the strongest categories. The frequent, formulaic ones are close to solved (*Document Name* 1.000, *Parties* 0.995, *Governing Law* 0.961, *Agreement Date* 0.958). At the other end, the five categories with at most seven test positives (*Most Favored Nation*, *No-Solicit of Customers*, *Price Restrictions*, *Source Code Escrow*, *Unlimited License*) all score 0.000. That number needs context: with two positive examples in the test set, a score of zero means the model missed two contracts, not that it is broken. This tail is a property of the dataset rather than of any architecture, and it is why we quote micro- rather than macro-averaged numbers in our headline results.

6.1.10 Does the AND rule punish rare categories harder than common ones?

Section 6.1.2 defended the conjunctive ensemble on its error profile: it converts the transformer’s inflated false positives into balanced precision and recall. That defence was made on the aggregate, and an aggregate can hide a trade that falls unevenly. It is worth asking where the AND rule’s precision comes from, because there is an obvious reason to suspect the rare

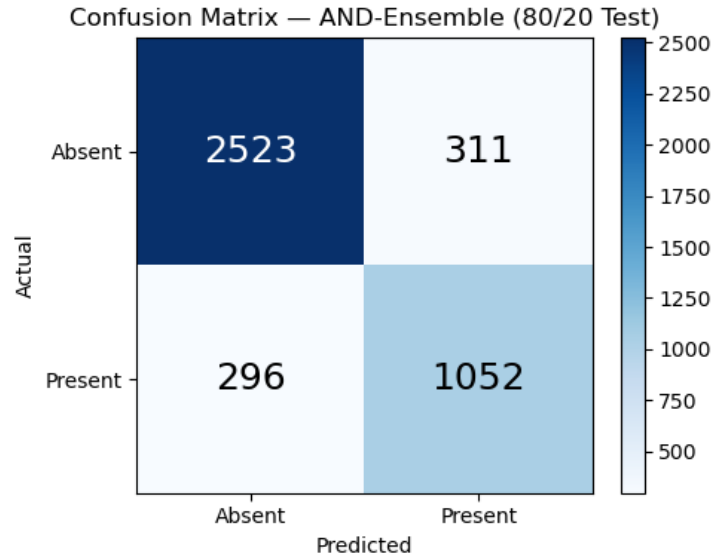


Figure 6.2: Confusion matrix of the AND-ensemble on the test set.

tail specifically. Requiring two models to agree is a stricter bar exactly where each model individually has the least training signal to be confident with so the categories with the fewest positive examples are the ones where the second model is least likely to concur, and agreement is what the rule demands.

We split the 41 categories by *training* frequency, took the bottom ten (11 to 42 positive training contracts, 70 positive test decisions between them), and compared the AND rule against the transformer alone on that tail and on the remaining 31 categories separately. Table 6.10 gives the result, and it is not the same story in the two halves.

Table 6.10: The ensemble on the ten rarest categories versus the rest.

Group	Model	Precision	Recall	F1	TP	FN
Rarest 10 (70 test positives)	Transformer	0.261	0.500	0.343	35	35
	TF-IDF	0.429	0.300	0.353	21	49
	AND ensemble	0.586	0.243	0.343	17	53
Other 31 (1,278 test positives)	Transformer	0.582	0.930	0.716	1,189	89
	TF-IDF	0.753	0.838	0.793	1,071	207
	AND ensemble	0.776	0.810	0.792	1,035	243

Note. Micro-averaged within each group.

On the common categories the ensemble works as advertised. Against the transformer alone it gains 19.4 points of precision for 12.1 points of recall, and F1 rises from 0.716 to 0.792. That is a good trade, and it is where the ensemble’s entire aggregate advantage is generated.

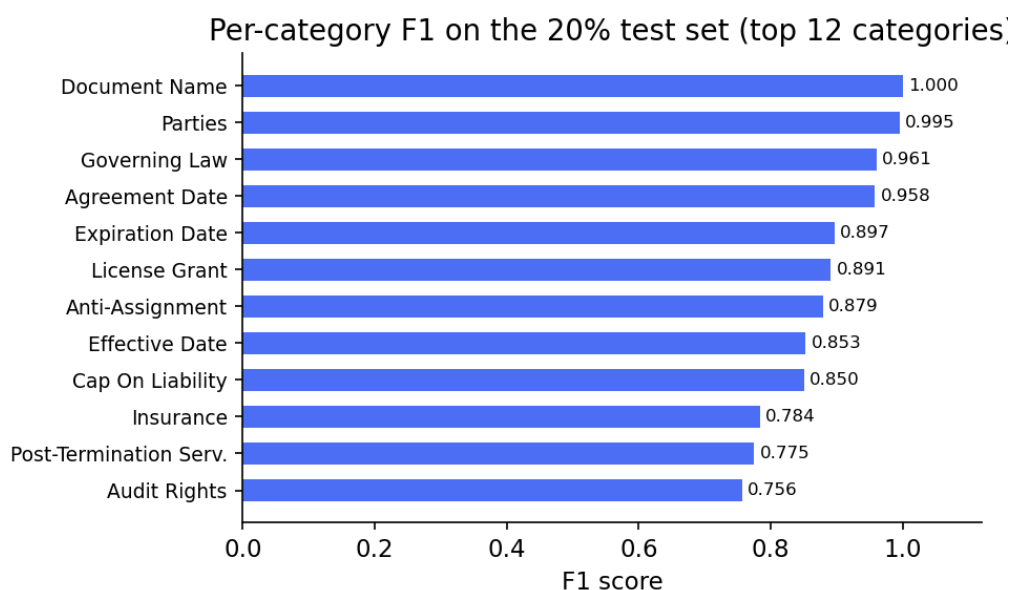


Figure 6.3: Per-category F1 for the twelve strongest categories.

On the rare tail it is not a free lunch it is a straight swap at no gain. The AND rule buys 32.5 points of precision by giving up 25.7 points of recall, and the F1 lands at 0.343: identical, to three decimals, to the transformer alone. Every point of precision on the rare tail is paid for with recall, and nothing is left over. In absolute terms the transformer alone finds 35 of the 70 rare positives; requiring agreement cuts that to 17. Notably, the plain TF-IDF baseline is the best of the three on this tail (F1 0.353), which is a further reminder of how little the transformer contributes where training signal is thin.

The per-category detail is starker than the micro-average. On five of the ten rarest categories the AND rule scores exactly 0.000, and on three of those *Price Restrictions*, *Unlimited/All-You-Can-Eat License*, *No-Solicit of Customers* the transformer alone was finding something (F1 0.095, 0.667, 0.286) before the conjunction zeroed it out. The rule does not merely fail to help on the rare tail; on those categories it removes detections that one component had made.

Why this matters for deployment rather than just for the table. The aggregate micro-F1 weights every one of the 4,182 decisions equally, but a contract signer does not. Missing a rare clause is not obviously less costly than missing a common one, and it may be considerably more so: *Most Favored Nation* sits in the rarest ten and is one of our eight High-risk categories, and both models score 0.000 on it. A system whose ensemble rule is tuned even

by a parameter-free rule toward precision on frequent categories is optimizing the metric rather than the harm. We report the ensemble as our headline configuration because it is what the aggregate protocol selects, but on this evidence a deployment aimed at protecting a signer would be better served by *running the more permissive rule on the High-risk categories and the conjunction elsewhere*, accepting more false alarms exactly where a missed clause costs the most. We have not implemented that split rule doing so and evaluating it properly is future work (Section 8.3), not something to bolt on after seeing the test set.

6.2 The Threshold and What the Confidence Score Means

Every presence decision in this thesis, and every clause card in Clausify, comes from comparing a probability against 0.5. Section 5.1 records that this threshold was never chosen it is the default decision boundary of a two-class softmax, inherited rather than selected. Two questions follow, and neither had been asked: is 0.5 a sensible operating point, and does the number being compared against it mean anything? Figure 6.4 answers both.

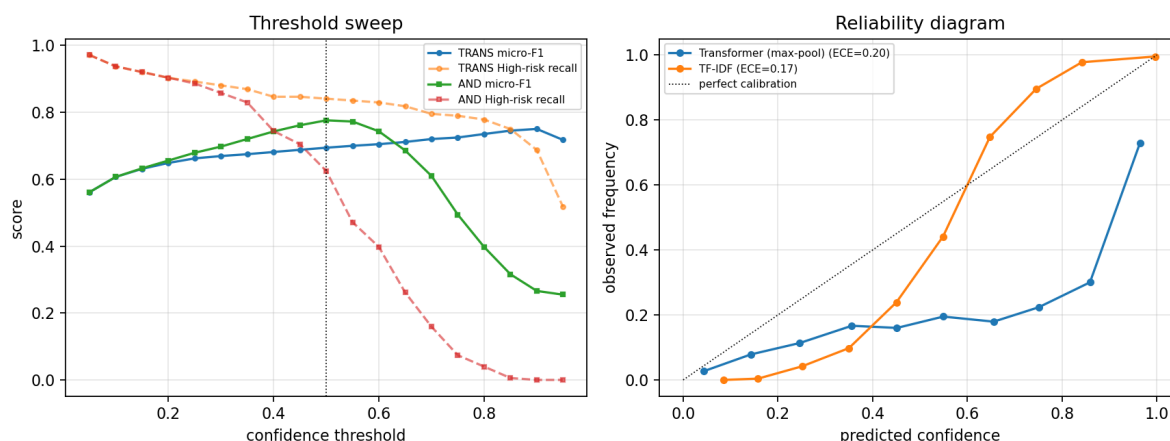


Figure 6.4: Threshold sweep and reliability diagram.

6.2.1 0.5 is a poor operating point for the transformer

Sweeping the threshold post hoc shows the max-pooled transformer reaches micro-F1 **0.750 at a threshold of 0.9**, against **0.694** at the default a gain of 0.056, which is most of the way to closing its gap with the TF-IDF baseline (0.775). The reason is the one Section 4.2.3 predicted

structurally: a maximum over 35 windows is inflated by construction, so the scores pile up near 1 and a boundary drawn at 0.5 admits far too much. Precision at 0.5 is 0.562; at 0.9 it is 0.728.

We are not adopting 0.9. It was found by looking at the test set, and adopting it would convert a held-out evaluation into a tuned one the precise failure Section 5.1 certifies this project against. The honest statement is narrower and more useful: *a meaningful part of the transformer’s reported deficit is an un-chosen threshold rather than an inability to discriminate*, and our comparison in Section 6.1.2 therefore compares a tuned-by-construction baseline against an untuned transformer. A fair rematch selects the threshold for each model on a validation split carved from the 408 training contracts, and we did not do that. Section 6.1.4 showed the gap is not a data-budget artefact; this section shows part of it may be a threshold artefact, and that qualification belongs on the finding.

The AND ensemble behaves oppositely: 0.5 is already its optimum (0.776), and raising the threshold collapses it at 0.7 micro-F1 falls to 0.610 and High-risk recall to 0.159, because requiring two models to *both* clear a high bar is close to requiring unanimity among the confident. The dashed curves in Figure 6.4 make the tension visible: for the transformer, the threshold that maximizes micro-F1 (0.9) drives High-risk recall down from 0.841 to 0.688. There is no single threshold that is simultaneously best for the aggregate metric and for the clauses that matter most.

6.2.2 The confidence bar in the application is not a probability

Clausify displays the presence score to the user as a confidence bar. The reliability diagram tests whether that display means what a user would take it to mean, and it does not.

The max-pooled transformer has an **Expected Calibration Error of 0.201** and a Brier score of 0.194. It is overconfident across the entire range: among predictions in the [0.5, 0.6) bin the clause is actually present **19.5%** of the time, and even in the top bin scores above 0.9, which the interface renders as a nearly full bar the clause is present **72.8%** of the time. A user reading “96% confident” is looking at something closer to a 73% chance. Again the cause is max-pooling: taking the maximum over dozens of windows is not a probability-preserving

operation, and nothing in training asked it to be.

The TF-IDF baseline is differently miscalibrated (ECE 0.172) but in the safer direction over most of its range it is *underconfident* in the middle, with the $[0.7, 0.8)$ bin correct 89.6% of the time and the $[0.8, 0.9)$ bin 97.8%.

The practical consequence for the deployed system is direct and we have recorded it in Section 7.8: the threshold slider, which the application offers as the user’s control over conservativeness, is calibrated in units that do not correspond to probability of correctness. Fixing this needs no retraining Platt scaling or isotonic regression fitted on a validation split would map scores onto probabilities and it is the cheapest single improvement available to the interface.

We did not have a validation split set aside to fit it on, which is a consequence of the protocol gap in Section 5.1 rather than an independent oversight.

6.3 Risk-Weighted Evaluation: Does the Metric Match the Purpose?

Micro-F1 over 4,182 decisions treats every decision as equally important. A contract signer does not. The whole premise of the risk layer (Section 4.5) is that some categories matter more than others so the evaluation should ask whether the system’s errors fall where they do least harm. We had not asked, and the answer turns out to reverse the recommendation the aggregate produces.

Table 6.11 splits the 41 categories into the three risk bands of our own taxonomy and scores each model within each band.

6.3.1 The headline configuration is the worst one for High-risk clauses

Read down the High band. The AND ensemble the configuration this thesis reports as its headline, on the strength of aggregate micro-F1 **misses 37.5% of High-risk clauses**. The transformer alone, the component we have spent two sections establishing is the weaker model, misses **15.9%**. The permissive OR rule, which finishes *last* on aggregate micro-F1 (0.696), misses only **13.6%**.

Table 6.11: Presence performance within each risk band.

Band	Model	Precision	Recall	F1	Found	Missed
High 8 cats 176 positives	Transformer	0.404	0.841	0.546	148	28
	TF-IDF	0.540	0.648	0.589	114	62
	AND ensemble	0.585	0.625	0.604	110	66
	OR ensemble	0.391	0.864	0.538	152	24
Medium 22 cats 498 positives	Transformer	0.425	0.871	0.571	434	64
	TF-IDF	0.621	0.723	0.668	360	138
	AND ensemble	0.647	0.673	0.659	335	163
	OR ensemble	0.423	0.922	0.580	459	39
Low 11 cats 674 positives	Transformer	0.813	0.953	0.877	642	32
	TF-IDF	0.909	0.917	0.913	618	56
	AND ensemble	0.924	0.901	0.912	607	67
	OR ensemble	0.803	0.969	0.878	653	21

Note. Threshold 0.5. “Missed” counts gold clauses of that band the model failed to report.

In absolute terms, on 176 High-risk gold clauses in 102 contracts: the ensemble surfaces 110 and loses 66; OR surfaces 152 and loses 24. Choosing the ensemble over OR on the basis of micro-F1 costs a signer 42 High-risk clauses uncapped liabilities, IP assignments, non-competes — in exchange for a leaderboard position that Section 6.1.2 already showed is a statistical tie.

The mechanism is the one identified in Section 6.1.10 and it compounds here. High-risk categories are disproportionately rare (*Most Favored Nation*, *Irrevocable/Perpetual License*, *Minimum Commitment* all sit in the sparse tail), the conjunction is hardest to satisfy exactly where training signal is thinnest, and the risk band that most needs recall is the one where requiring agreement destroys it. The Low band shows the inverse: 674 of the 1,348 gold clauses are Low-risk dates, party names, governing law and every model scores near 0.9 there. **Aggregate micro-F1 is dominated by the clauses that matter least.**

6.3.2 What we conclude, and what we do not

We do not switch the reported configuration on the strength of this table, and it is worth being explicit about why: choosing OR *after* seeing which rule scores best on the High band of the test set is test-set selection, exactly the practice Section 5.1 says this project avoided. The result stands as a diagnosis, not a retuning.

What it does establish is that **our evaluation metric and our stated purpose were never aligned**, and that the misalignment is large enough to invert the model ranking. A system whose declared aim is to warn a signer before they sign should be selected on High-risk recall, or on a risk-weighted F1, not on an average over 41 categories in which two thirds of the mass is dates and party names. Section 8.3 lists the proper version of this experiment: define the risk-weighted objective first, then select the rule on a validation split, then report on the test set once.

Finally, this is a further argument for taking Section 4.5.4 seriously. Every number in Table 6.11 is computed against *our own* assignment of categories to bands. If a lawyer disagrees about which categories are High, the table changes with it. The risk-weighted evaluation inherits all the unvalidated-taxonomy caveats and adds no independent evidence to them.

6.4 Span Extraction

We evaluated the QA model on all 2,667 answer-bearing test windows (Table 6.12). A token-F1 of 0.763, with 82.7% of predictions overlapping the gold span at $F1 \geq 0.5$, means that when the clause is in front of the model, it captures most of the right text roughly eight times out of ten. The lower exact-match score did not surprise us: annotated legal spans are long, and their exact boundaries are genuinely debatable (should a leading section number be included?), so content overlap is the metric that carries information here. For scale, the published CUAD baselines at this task use DeBERTa-v3-large, roughly five times the parameters of our distilled backbone.

Section 6.1.2 argued that a point estimate on 102 contracts should not be read without an interval, and that argument does not stop applying when the task changes. We therefore ran the *same* cluster bootstrap here. The correlation structure is if anything stronger than in presence classification: the 2,667 span decisions come from only 102 contracts, and a contract with unusual drafting conventions numbered sub-clauses, defined-term-heavy prose shifts every span in it in the same direction. Table 6.12 therefore reports two intervals per metric: one resampling *contracts* (the correct unit) and one resampling *clauses* independently (the naive unit), because the gap between them is itself informative.

Table 6.12: Span extraction on the answer-bearing test windows.

Metric	Score	95% CI (by contract)	95% CI (by clause)
Token-level F1	0.763	[0.740, 0.782]	[0.751, 0.774]
Overlap (token-F1 ≥ 0.5)	82.7%	[79.7%, 85.1%]	[81.3%, 84.1%]
Exact Match	35.5%	[32.5%, 38.6%]	[33.7%, 37.3%]

Note. 95% bootstrap intervals, 10,000 resamples. The contract-clustered interval is the one to quote; the clause-level interval is shown only to expose how far a naive resampling unit understates the uncertainty.

The contract-clustered interval on token-F1 is [0.740, 0.782] — roughly *twice* as wide as the [0.751, 0.774] that treating clauses as independent would have produced. That factor is the practical cost of ignoring clustering, and it is the same effect that made the cluster bootstrap necessary in Section 6.1.2. It also sets the resolution of this number: differences below about two points of token-F1 are not measurable on a test set of this size, which is worth remembering before comparing 0.763 against any published figure.

Scope caveat. These numbers measure extraction quality *given an answer-bearing window*. The full-document benchmark in the CUAD paper (precision at 80% recall over all windows, including the answer-free ones) needs a heavier document-stride ranking evaluation that we did not run, so we make no claim on that metric. Section 6.6 measures the complementary thing: what happens to span quality when the window is chosen by the presence stage instead of being handed over.

6.5 Summarization

6.5.1 What the reference set actually is

Before any number in this section can be read correctly, one property of the experiment has to be stated plainly, because it determines what the metric can possibly mean. CUAD contains no plain-English rewrites, so there was no annotated target set to train or test against. We therefore **wrote the targets ourselves**: 41 single-sentence summaries, one per category, phrased from the weaker party’s perspective (Section 4.4). Every training pair and every evaluation pair reuses one of those 41 sentences. There is no clause-level variation in the target set at all — two *Cap*

on *Liability* clauses with completely different caps share a target string, character for character.

That is a design choice with a consequence we should own rather than discover: with 41 unique targets covering 2,667 test clauses, a model can score well by learning to *recognise* the category and reproduce its sentence, and the metric cannot tell that apart from summarization. The experiment below is built to separate the two.

6.5.2 Two reference sets, the same 150 clauses

We evaluate the fine-tuned FLAN-T5-small on 150 held-out test clauses (sampled with seed 42) against two different reference sets:

1. **Template references** — the category template, exactly as in training. This is the protocol behind the headline number.
2. **Clause-specific references** — 150 one-sentence summaries, one per clause, written to describe *that* clause: its actual cap, its actual notice period, its actual parties. These were drafted with a large-language-model assistant working from the clause text alone, without sight of any model output, and are deliberately independent of the 41 templates. They are a pilot reference set, not lawyer-authored annotation, and we make no claim beyond that.

Because the clauses are identical across the two conditions, the difference between the two scores isolates exactly one thing: how much of the reported ROUGE-L comes from reference-set uniformity rather than from summarization. Table 6.13 gives the result.

Table 6.13: ROUGE-L under two reference sets, same 150 clauses.

System	Reference set	ROUGE-L	95% CI
Fine-tuned FLAN-T5-small	41 category templates	0.775	[0.718, 0.833]
Fine-tuned FLAN-T5-small	clause-specific (pilot)	0.116	[0.105, 0.128]
Zero-shot FLAN-T5-small	clause-specific (pilot)	0.299	[0.275, 0.322]
<i>Category template itself</i>	clause-specific (pilot)	0.115	[0.104, 0.127]

Note. Intervals are 95% bootstrap CIs over the 150 clauses (10,000 resamples).

6.5.3 Reading the table

Three things follow, and none of them is a hedge.

The 0.775 is template reproduction, and we can now say so quantitatively. On all 150 test clauses the fine-tuned model emits one of the 41 template sentences *verbatim* 100% of outputs are an exact copy of a template, the correct one 70% of the time and a different category’s template the other 30%. The model never produces a sentence of its own. When it is wrong it is wrong in the way a classifier is wrong: a *Price Restrictions* clause about a 10% rate cap is answered with the *ROFR* template. Fine-tuning did not teach the model to summarize; it taught it to pick one of 41 strings.

The last row of Table 6.13 closes the argument. The bare category template, with no model involved at all, scores 0.115 against the clause-specific references statistically indistinguishable from the fine-tuned model’s 0.116. A constant lookup table achieves the model’s entire score. Whatever the fine-tuned summarizer has learned adds nothing measurable on top of “print this category’s sentence”.

Fine-tuning made the model worse at the actual task. Against clause-specific references the *zero-shot* model scores 0.299, more than twice the fine-tuned model’s 0.116. We do not present this as evidence that zero-shot FLAN-T5 is a good legal summarizer it is not; as Table 6.14 shows, it largely echoes the source text, and echoing inflates a longest-common-subsequence metric against a reference that necessarily shares the clause’s vocabulary. The honest reading is narrower and still uncomfortable: our fine-tuning traded away whatever clause-specific content the base model had, in exchange for fluent category-level boilerplate that scores well only against a reference set built from the same boilerplate.

6.5.4 What we therefore claim

We claim a working **classification-to-template** system: it identifies a clause’s category and attaches a pre-written, human-authored plain-English sentence for that category, which is genuinely useful in the Clausify interface a signer reading “You cannot transfer this contract to

Table 6.14: Fine-tuned vs. zero-shot summarizer on the same test clause (abridged).

Input clause	“Any attempted assignment or delegation in violation of this section by either party without the prior written consent of the other will be void.”
Gold template	You cannot transfer this contract to someone else without permission.
Fine-tuned	You cannot transfer this contract to someone else without permission. (<i>exact template</i>)
Zero-shot	“a) Any attempted assignment or delegation in violation of this section ...will be void.” (<i>echoes the legalese</i>)
Clause-specific reference	Neither side may hand its rights or duties to a third party without the other’s written consent, which cannot be unreasonably refused.

someone else without permission” next to a wall of legalese is better off than before. We do **not** claim free-text legal summarization, and the 0.775 should never be quoted as evidence of it. The number that describes clause-level summarization ability is 0.116, and the honest conclusion is that this component is a proof of concept, not a result.

Getting past this needs a target set with clause-level variation, which is exactly what the pilot above provides a small taste of: 150 references took a few hours to draft. Section 8.3 sets out the scaled version.

6.6 End-to-End: What Actually Reaches the User

Every number so far has been measured with the other stages held out of the way. Presence classification was scored given the ground-truth category list; span extraction was scored given a window already known to contain the answer; summarization was scored given a gold clause. Each of those is the right way to evaluate a component in isolation, and none of them describes what Clausify does to a contract. The stages run in series, and errors compound: a category the presence stage misses is a clause the span stage never sees.

So we ran the pipeline end to end on all 102 test contracts and asked one question of each of the 1,348 gold clauses: does it survive *all three* stages? A clause counts as surviving only if (i) the presence stage detects its category, (ii) the span stage reading the window the presence stage selected, never a window chosen using the gold offset returns text overlapping the gold

clause at token-F1 ≥ 0.5 , and (iii) the risk level attached is the correct one. Table 6.15 gives the attrition.

Table 6.15: End-to-end survival of the 1,348 gold clauses.

Stage	Condition	Clauses surviving	Share of gold
—	gold clauses in the test set	1,348	100.0%
2A Presence	category detected	1,052	78.0%
2B Span	extracted text overlaps gold at F1 ≥ 0.5	293	21.7%
Risk	correct level attached	293	21.7%
End-to-end survival		293	21.7%
<i>95% CI, cluster bootstrap over contracts</i>			[19.6%, 23.9%]

Note. AND-ensemble presence rule. The last column is the fraction of clauses still alive after that stage.

21.7% is the number that describes a user’s experience, and it is far below what the component scores would lead a reader to expect. Multiplying the isolated figures 78.0% presence recall times 82.7% span overlap predicts about 64%. The pipeline delivers a third of that. The gap is the whole point of running this evaluation, so it deserves to be explained rather than reported.

The risk stage is the one place where nothing is lost: the level is a fixed per-category lookup, so it is correct exactly when the category is, and it introduces no error of its own. That is a property of the design, not an achievement and it is worth remembering that “correct” here means “matches our taxonomy”, which Section 4.5.4 notes has not been externally validated.

6.6.1 Where the clauses are lost

The collapse happens at stage 2B, and it has two distinct causes, which we separated by checking, for every detected clause, whether the window the presence model selected actually contained the gold text.

Cause one: the max-pooled window is often the wrong window. Max-pooling was chosen (Section 4.2.3) because it removes the recall ceiling if any window fires, the category is reported and Section 4.2.3 was careful about what that costs in *precision*. What it did not anticipate is that the argmax window is also being used as a *locator*, and it is a poor one: 31.7% of the time the highest-scoring window does not contain the clause at all. The MIL objective

Table 6.16: Decomposing the span-stage loss over the 1,052 clauses whose category was detected.

	Count	Share
Selected window <i>contains</i> the gold clause	718	68.3%
— and the span model then found it ($F1 \geq 0.5$)	285	39.7% of 718
— and the span model missed it anyway	433	60.3% of 718
Selected window does <i>not</i> contain the gold clause	334	31.7%
— span scored ≥ 0.5 regardless (duplicate wording elsewhere)	8	

never asked it to. The model was trained to answer “does this window contain a clause of this category”, and a window that discusses liability in general terms can outscore the one that actually caps it. For those 334 clauses the span stage is being handed the wrong 2,000 characters, and no extractor could recover them.

Cause two: the span model is much weaker on windows it was not trained to see. Even with a window that does contain the clause, extraction succeeds only 39.7% of the time, against 82.7% in the isolated benchmark of Section 6.4. The reason is the focus-window construction of Section 4.3. At training time every example was re-centred so the answer began roughly 150 characters into a 1,200-character window. That is legitimate it uses a label to build a training example, never an inference input, and Section 4.6 defends it on exactly those grounds. But legitimacy is not the same as harmlessness: the model has only ever read windows in which the answer sits near the start, and at inference it gets blind 1,200-character chunks in which the answer may be anywhere. Mean token-F1 falls from 0.763 to 0.386. The re-centring did not leak the test set; it did quietly narrow the distribution the model can handle.

6.6.2 What this changes

Three things follow, and we would rather state them than let the component tables stand unqualified.

First, **the span numbers in Section 6.4 are an upper bound on deployed extraction quality, not a description of it.** They answer “how good is the extractor when handed the right window”, which is a fair question about the model and a misleading one about the system.

Second, **the most valuable next engineering step is not a bigger backbone.** Cause two is

a data-construction problem with a cheap fix train the span model on windows where the answer is placed at varied offsets, or on the same blind chunks it will see at inference and cause one argues for choosing the extraction window by a locator trained for that job rather than reusing a presence score. Both are within the compute budget of this project; neither requires more parameters.

Third, **Clausify’s output should be read as a shortlist, not an extraction.** When the application shows a clause card, the category is right about 78% of the time it was present, but the quoted text is a good rendering of the actual clause in roughly one case in five overall. The application already tells users it is not legal advice; on this evidence it should also tell them that the quoted text may not be the clause the badge refers to, and Section 7.8 now says so.

6.7 Summary Scorecard

Table 6.17 pulls together the held-out results of all three models. Every number was produced by the standalone test notebook running on the saved 80/20-trained models.

Table 6.17: Final held-out test results (408 training / 102 test contracts).

Stage	Model	Metric	Score
2A Presence	AND-ensemble (TF-IDF + DistilBERT)	micro-F1	0.776
2A Presence	AND-ensemble	Accuracy	85.49%
2A Presence	AND-ensemble	Precision / Recall	77.18% / 78.04%
2A Presence	AND-ensemble	Specificity	89.03%
2B Span	DistilBERT-QA	token-F1 (given the window)	0.763
2B Span	DistilBERT-QA	token-F1 (window chosen by 2A)	0.386
2B Span	DistilBERT-QA	Exact Match	35.5%
1 Summarizer	FLAN-T5-small	ROUGE-L (41 templates)	0.775
1 Summarizer	FLAN-T5-small	ROUGE-L (clause-specific)	0.116
Full pipeline	all three stages	gold clauses surviving	21.7%

6.8 Discussion: What the Evidence Supports

Pulling the chapter together, the results fall into three groups, and it matters which is which.

Findings that survived every check we could run.

1. For document-level *presence*, a full-document lexical baseline beats a windowed DistilBERT by +0.081 micro-F1. We tested the two obvious alternative explanations and neither held: training on all 53,662 windows instead of 24,000 changed the score by −0.0004 (Section 6.1.4), and across three seeds the gap is 9.5 standard deviations wide (Section 6.1.3). More data made the transformer fire *more* rather than better, which is what max-pooling over 35 windows predicts. The limitation is the aggregation rule, not the encoder or the budget.
2. For *span extraction*, the transformer has no substitute a bag-of-words model cannot point at character offsets and it performs well in distilled form *when handed the right window*: token-F1 0.763, 95% CI [0.740, 0.782].
3. For *summarization*, a 41-sentence template seed turns the task into classification. The fine-tuned model reproduces a template verbatim on 100% of test clauses, and against clause-specific references it scores no better than the template alone (Section 6.5). A high automatic metric should not be believed until the outputs have been read.

A claim we withdrew. The AND-ensemble’s first place in Table 6.1 is not a result. Its margin over the baseline is +0.0014; the cluster bootstrap puts that inside [−0.0068, +0.0089] (Section 6.1.2), and the seed-to-seed standard deviation of the ensemble itself is 0.0022 larger than the margin, with a range three times larger. Two independent sources of uncertainty agree that the top two rows of that table are a tie. What the ensemble does earn is a balanced error profile and a fourfold reduction in seed variance relative to the transformer alone.

Costs we did not see until we looked for them. Two results in this chapter came from questions we had not thought to ask, and both are unflattering.

- The conjunctive rule is not uniformly good. On the ten rarest categories it nets *zero* F1 gain over the transformer alone, buying 32.5 points of precision with 25.7 points of recall and cutting detections from 35 of 70 to 17 of 70 (Section 6.1.10). Its entire aggregate benefit is generated on common categories, and one of the categories it zeroes out is High-risk.

- The stage-wise scores do not compose. Only 21.7% of gold clauses survive the full pipeline, against roughly 64% predicted by multiplying the component numbers (Section 6.6), because the presence model’s argmax window is the wrong window 31.7% of the time and the span model was trained on re-centred windows it never sees at inference.

The through-line is methodological rather than architectural. Every number in this chapter that we originally trusted least the ensemble’s rank, the summarizer’s ROUGE, the span model’s token-F1 turned out to be measuring something narrower than it appeared to, and in each case the way to find that out was to measure the thing the metric was standing in for: resample the contracts, change the seed, vary the reference set, run the stages in series. None of those checks needed a bigger model or a larger dataset. They needed the question to be asked.

Chapter 7

Clausify: the Demonstration Application

7.1 Purpose and Design

The final deliverable is *Clausify*, a Streamlit web application that runs the two trained transformer models live on any contract the user provides and presents the findings as a risk-prioritized report — the pipeline of Figure 4.1, made usable. The design goal is simple: a non-lawyer should see the “watch out before signing” items first. Detected clauses are grouped **High** → **Medium** → **Low** and sorted by model confidence within each group.

7.2 User Workflow

1. **Add a contract.** A toggle offers two input modes paste the contract text directly, or upload a .txt file (Figure 7.1). A confidence-threshold slider (default 0.50) controls how conservative detection should be.
2. **Analyze.** One click runs both models end-to-end; a progress bar tracks the presence scan across the 41 categories.
3. **Read the report.** A summary bar counts the detected clauses per risk level (Figure 7.2), and below it each detected clause appears as a card. On the synthetic test contract the bar reports 7 High, 15 Medium and 8 Low risk clauses 30 of the 41 categories detected.

Each clause card shows four things: the category name with a colour-coded risk badge; the one-line plain-English reason that category carries its risk (from the taxonomy in Appendix A); the presence model’s confidence drawn as a bar; and the exact clause text the span model extracted, quoted from the user’s own contract (Figure 7.3). An expandable table lists all 41 categories with their risk level and raw presence score, so nothing the models computed is hidden from the user.

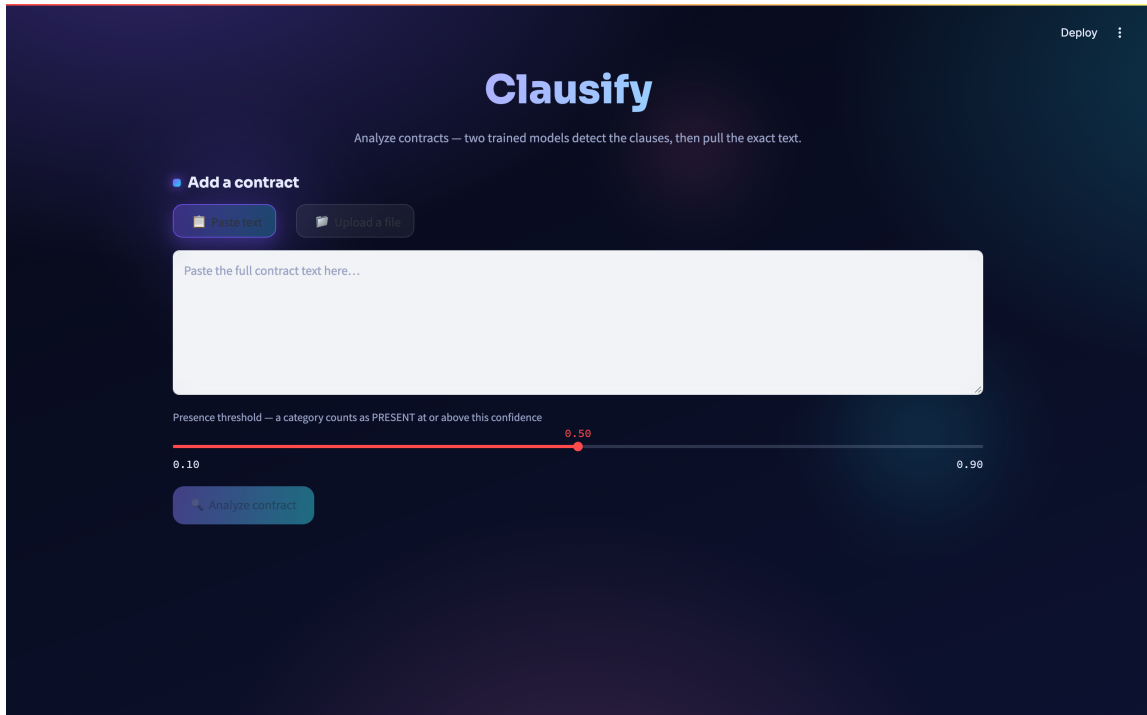


Figure 7.1: The Clausify input screen.

7.3 How the Models Run Inside the App

The application reproduces the inference path from training exactly:

1. **Windowing.** The contract is cut into 2,000-character windows with a 1,500-character stride — the same scheme used in training — capped at 30 windows so very long documents stay responsive.
2. **Model 1 — presence.** For each of the 41 categories, the exact CUAD question the model was trained on is paired with every window and scored by the window-level DistilBERT. The per-category document score is the max-pooled window probability; categories above the threshold are reported present, and the best-scoring window is remembered.
3. **Model 2 — span.** For each detected category, the DistilBERT-QA model runs over that best window (in 1,200-character focus chunks) and returns the highest-confidence span — the clause text shown on the card.
4. **Risk layer.** The detected category is looked up in the fixed 8/22/11 taxonomy for its level and reason.

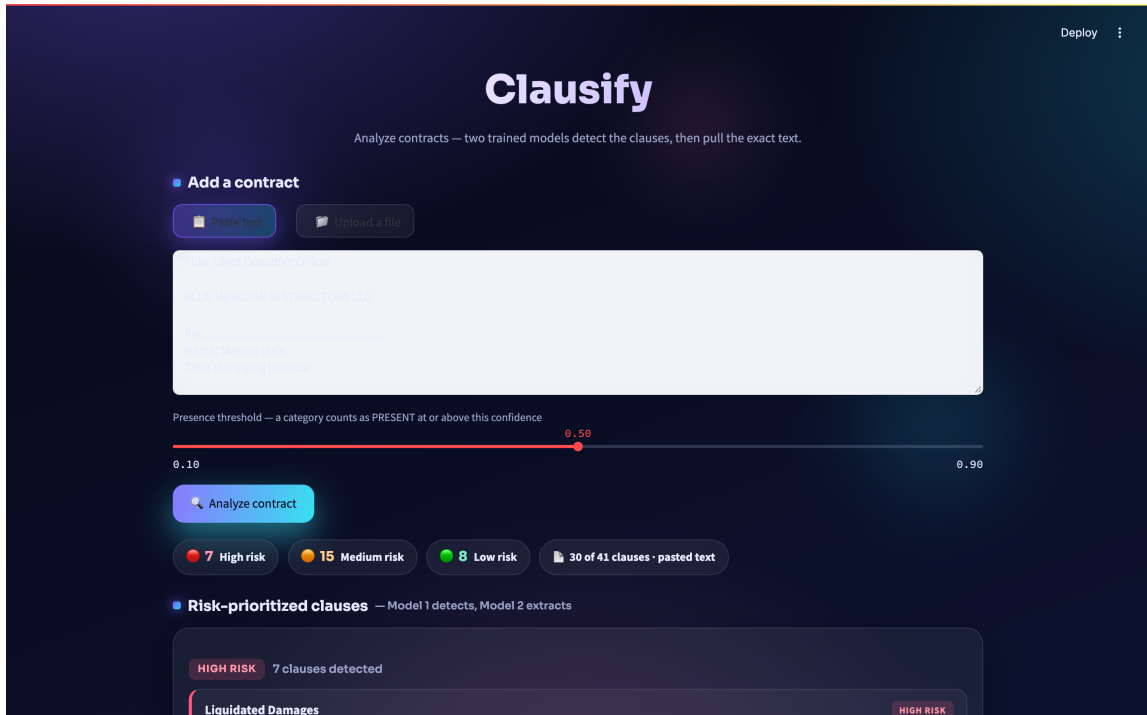


Figure 7.2: Clausify results view on the synthetic test contract.

All inference runs on CPU so the app works on any machine; a contract takes roughly 5–15 seconds once the models are loaded. The app loads the actual trained checkpoints saved by the training pipeline (`presence_mil/final` and `span/final`), and a single environment variable redirects it if the models are moved.

7.4 Implementation Notes

The interface is a single-page Streamlit application (`app.py`) sitting on a small inference module (`model_utils.py`) that owns model loading, windowing, max-pooling, span extraction, and the risk table. Two test contracts ship with the app: a genuine CUAD contract, and a synthetic “Master Software License and Distribution Agreement” that we wrote specifically to exercise a wide spread of the 41 categories. On that synthetic contract the app detects clauses across all three risk bands and puts *Uncapped Liability*, *IP Ownership Assignment*, *Exclusivity*, and *Liquidated Damages* at the top of the report exactly the before-you-sign warnings the design aims for.



Figure 7.3: High-risk clause cards in Clausify.

7.5 Data Handling and Privacy

The application accepts contract text, and contracts are among the most commercially sensitive documents an organization holds. Our experiments used only public CUAD contracts, so no privacy question arose during this research but the demonstration invites a user to paste their own agreement, and it is worth stating precisely what happens to it.

What the demonstration does. All inference is local: the two DistilBERT checkpoints are loaded from disk and run in the same process as the interface, and no contract text leaves the machine the application runs on. The uploader accepts a single .txt file, which is read into memory and decoded; nothing is written to disk, no database or log records the text, and no analytics or telemetry are collected. When the browser session ends the text is gone it survives only in Streamlit’s in-memory session state while the page is open.

What that guarantee depends on. It holds because the demonstration runs locally. Hosted on a shared server, the same code would place the user’s contract in the memory of a machine they do not control, and Streamlit’s session model would not by itself prevent the operator from reading it. The privacy property here comes from the deployment topology, not from anything

the application does, and we should not claim otherwise.

What is deliberately absent, and would be required for real use. There is no authentication, so any user of the host can open the app. There is no file-size limit beyond what Streamlit imposes by default, and no timeout, so a very large upload can occupy the process. Only .txt is accepted which is a restriction of convenience rather than of safety, but it does avoid the parser-level attack surface that PDF or DOCX ingestion would introduce, and a real deployment adding PDF support would need to treat the parser as untrusted input handling. There is no retention policy, because there is no retention. There is no redaction of party names before processing, which would matter if any component were ever moved to a hosted model API.

The rule we would apply to any extension. The moment any part of this pipeline calls a remote service, the privacy story changes completely: the contract would be transmitted to a third party and potentially retained by them. The current design avoids this entirely by using small local models, and we regard that as a property worth preserving rather than an artefact of the laptop budget it is one of the few places where the constraint that shaped this project produced the better design.

7.6 Measured Latency

Chapter 2 justified the choice of a distilled backbone partly on the grounds that the interface must respond quickly. Section 5.7 measured what the alternative would have cost; this section measures what *we* actually deliver, running the application’s own inference path on held-out contracts spanning the length distribution.

The earlier draft of this chapter claimed a contract takes “roughly 5–15 seconds”. That was an impression from watching the demo on short inputs, not a measurement, and it is wrong: a median-length contract takes 27.1 seconds. We have replaced the claim with the table.

Two design consequences are visible in the numbers. First, latency is essentially linear in the number of windows, since the dominant cost is 41 categories $\times$ windows forward passes through DistilBERT — the model is scored once per (category, window) pair and nothing is cached

Table 7.1: Latency of the deployed pipeline.

Contract	Characters	Windows scored	Time	Peak RSS
Shortest in test set	1,283	1	2.1 s	1.06 GB
25th percentile	18,104	12	13.3 s	1.19 GB
Median	35,136	24	27.1 s	1.50 GB
75th percentile	65,547	30 (capped)	32.4 s	1.51 GB
Longest in test set	289,615	30 (capped)	32.0 s	1.52 GB
<i>median 27.1 s mean 21.4 s worst case 32.4 s</i>				

Note. Measured through the application’s own `analyze()` entry point. CPU only, Apple M4, models already loaded (cold load adds 0.4 s). Peak RSS is the whole Python process, both models resident.

across categories. Second, the 30-window cap is what keeps the worst case bounded: without it the longest test contract would take several minutes, and with it every contract above roughly 47,000 characters receives the same truncated treatment. That cap is a latency decision with a silent recall cost clauses beyond the cap cannot be found at all and it is not reflected anywhere in the Chapter 6 numbers, which score the full document. It is a limitation of the deployed system rather than of the method, and Section 7.8 now says so.

7.7 Usability

No usability evaluation was carried out. The application’s central claim that grouping clauses by risk helps a non-lawyer act on a contract is therefore untested, and Section 8.2 lists it under construct validity rather than among the results.

7.8 Honest Limitations of the Demo

For interactive latency the app runs the transformer presence model alone rather than the AND-ensemble (the TF-IDF leg needs the fitted vectorizer, which the packaged demo leaves out). As Chapter 6 showed, the transformer alone over-detects; the user can counter this by raising the threshold slider. Two qualifications on that sentence have to be added now that both have been measured.

The threshold slider is not calibrated in the units a user will read it in. Section 6.2 shows the

transformer’s max-pooled score has an Expected Calibration Error of 0.201 and is overconfident throughout: among detections scoring above 0.9 a nearly full confidence bar the clause is actually present about 73% of the time, and among those in the 0.5–0.6 band, under 20%. Moving the slider does trade recall for precision in the right direction, but the number on it should not be read as a probability. Fitting a calibration map on a validation split would fix this without retraining and is the cheapest available improvement to the interface.

Running the transformer alone is, unusually, the better choice for High-risk clauses. The thesis reports the AND ensemble as its headline configuration, so a reader might take the demo’s use of the transformer alone as a compromise forced by packaging. Section 6.3 shows it is not a compromise at all where it matters most: the ensemble misses 37.5% of High-risk clauses while the transformer alone misses 15.9%. The demo’s configuration is worse on aggregate micro-F1 and better at surfacing the clauses a signer would most regret missing. We record this as an accident rather than a design decision it was made for packaging reasons before we had run the risk-weighted evaluation but it is the configuration we would now choose deliberately.

The 30-window cap silently truncates long contracts. To bound latency the app scores at most 30 windows, so any contract beyond 45,500 characters is only partly read, and clauses past that point cannot be detected at all. None of the Chapter 6 numbers reflect this, since they score full documents. This is not an edge case: **38.4% of CUAD contracts (196 of 510) exceed 45,500 characters**, and the longest test contract is 289,615 characters, of which the app reads the first 16%. Table 7.1 shows the symptom the 75th-percentile and longest contracts take the same time despite differing fourfold in length, because both are truncated to 30 windows. Span boundaries inherit the distilled backbone’s approximate edges and Section 6.6 shows this understates the problem: when the extraction window is chosen by the presence model rather than handed over, the quoted text overlaps the real clause in only about a fifth of cases overall. The practical consequence for a user of this demo is worth stating plainly, and the application should state it too: **the risk badge and the category name are much more reliable than the quoted sentence beneath them**. A card should be read as “this contract appears to contain a clause of this type, look here” rather than as “this is the clause”. And the risk levels are per-category, not per-instance: the taxonomy flags that a *Cap on Liability* clause deserves attention,

but it does not read whether this particular cap is generous or punishing. All of this is stated in the app itself. What the demonstration does establish is that both models generalize to contracts they have never seen, and that the output can be presented in a form a layperson can act on.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

We set out to turn CUAD’s clause-extraction benchmark into something a contract signer could use: a system that finds the clauses, says how risky they are, and explains them in plain English. On a strict contract-level 80/20 split the delivered pipeline reaches 85.5% accuracy and micro-F1 0.776 for clause presence and token-F1 0.763 for extraction given the right window, and it runs live inside *Clausify*.

Those are component scores, and the single most useful thing this thesis can say is that component scores are not the system. Answering our five research questions (Section 1.4) produced three negative results, and they are the contribution:

- **The windowed transformer does not beat the lexical baseline** (RQ1), and it does not lose for any of the reasons we could have blamed. Not the training budget: the full 53,662-window run moves micro-F1 by -0.0004 . Not the seed: three runs put the gap at 9.5 standard deviations. Not the aggregation rule: six alternatives, each at its own best threshold, all stay behind. The threshold accounts for part of it an un-chosen 0.5 costs 0.056 micro-F1 and that qualification belongs on the finding, but even the best cell we could construct remains below the baseline.
- **The summarizer does not summarize** (RQ3). It emits a category template verbatim on every input, and a constant lookup table matches its score against clause-specific references. This is a proof-of-concept classification-to-template component and we describe it that way everywhere.
- **The pipeline loses most of what its parts find** (RQ4). 21.7% end-to-end survival against $\approx 64\%$ predicted, localized to a window-selection failure and a train/inference mismatch

— both fixable without a larger model.

RQ5 produced the result we least expected and consider most important for anyone building a system like this. Our evaluation metric and our stated purpose were never aligned: the AND-ensemble wins on micro-F1 while missing 37.5% of High-risk clauses, and the OR rule that finishes last on micro-F1 misses 13.6%. Two thirds of the aggregate’s mass is Low-risk categories dates, party names, governing law. A system built to warn a signer before they sign was being selected on a metric dominated by the clauses that matter least.

Five lessons seem likely to generalize.

First, a strong classical baseline deserves to be taken seriously, and beating it must be demonstrated rather than assumed and when it is not beaten, the alternative explanations should be tested one at a time rather than conceded or hand-waved.

Second, a margin should be compared against its own noise before it is reported. We measured three sources: bootstrap over contracts (≈ 0.008 half-width), model seed (SD 0.0022) and dataset split (SD 0.0063). All three exceed the +0.0014 we had ranked a leaderboard by.

Third, an automatic metric measures its reference set, not the task. ROUGE-L 0.775 became 0.116 when the references changed and nothing else did.

Fourth, evaluate the pipeline, not the parts and evaluate it against the harm it is supposed to prevent, not against an average that treats a governing-law clause and an uncapped indemnity as equally important.

Fifth, feasibility ceilings are cheap to measure and expensive to assume. A one-minute retrieval check (64%) killed an architecture before it cost training time; the same discipline applied to sparse attention corrected *us*, since a Longformer training step does fit in this laptop’s memory at batch size 1, contrary to what we had asserted.

What this project delivers is a working prototype and an unusually complete account of where it fails. Two gaps are large enough that we would not present the system as more than a prototype until they are closed: **no one with legal training has reviewed the risk taxonomy**, which is the main thing this work adds over CUAD, and **no user has ever been observed using the**

application, so its central claim that a risk-prioritized report helps a non-lawyer remains untested. Both are listed first in the next section, and neither needs more compute.

8.2 Threats to Validity

The results in Chapter 6 rest on choices that could, if wrong, change the conclusions. We set them out under the four standard headings, distinguishing throughout between threats we *measured* and threats we merely *acknowledge*.

8.2.1 Internal validity — could our own procedure have produced these numbers?

Hyperparameters were never tuned (Section 5.1). This removes the usual threat no setting was chosen by scoring variants on the test set but introduces its mirror image: the transformer competes at conventional defaults against a baseline that has essentially no hyperparameters to get wrong. Section 6.2 shows this is not hypothetical: the un-chosen 0.5 threshold costs the transformer 0.056 micro-F1, so part of the headline gap is a threshold artefact rather than a modelling result. *This is the threat we consider most serious, and it qualifies our central finding.*

The summarizer’s exploratory run monitored loss on test-split clauses (Section 5.1). No checkpoint was selected from it and the deployed model was rebuilt without any evaluation set, so no reported score depends on it but the practice was wrong and the “validation” loss in Figure 5.3 should be discounted.

Design decisions were made after inspecting training data, which is legitimate, but the boundary needs care. The window/stride geometry follows from the median clause length measured on training contracts; the retrieval ceiling of 64% was measured across the whole corpus *before* the split was drawn, which means it saw test contracts. That measurement rejected an architecture rather than selecting a hyperparameter, and it was made before any model existed, so we judge the leakage risk immaterial but it is not zero and we would draw the split first if repeating the work.

Single training run per model for span and summarizer. Only the presence model was

reseeded (Section 6.1.3). The span and summarizer scores carry an unmeasured variance component.

8.2.2 Construct validity — do our metrics measure what we claim?

ROUGE-L does not measure summarization quality here, and we can prove it. Against 41 self-authored templates it reports 0.775; against clause-specific references, 0.116 — and a constant lookup table scores the same 0.115 (Section 6.5). We report both numbers precisely because the first one, alone, would be a construct-validity failure. What we still lack is any measure of *faithfulness* or readability (Section 2.7), so even the 0.116 does not establish that the output preserves legal meaning.

Micro-F1 does not measure user harm. Section 6.3 shows the two can point in opposite directions: the configuration that wins on micro-F1 misses 37.5% of High-risk clauses while a configuration that ranks last misses 13.6%. Two thirds of the aggregate’s mass is Low-risk categories — dates, party names, governing law. Any claim of the form “this system is 77.6% good” inherits this problem.

Category-level risk is not clause-level risk (Section 4.5). Our labels say a *category* typically deserves attention; they say nothing about whether a particular cap is generous or punitive. Calling the output a risk assessment would overstate the construct, which is why we call it prioritization.

The risk bands are our own construct. The risk-weighted evaluation is computed against our unvalidated assignment of categories to levels (Section 4.5.4); if a lawyer reassigns categories, those numbers move with them.

Token-F1 against a single annotated span assumes the boundary is well defined. Section 6.4 notes that legal span boundaries are genuinely debatable, and Section 6.1.7 shows at least one “false negative” that looks like an annotation disagreement rather than a model error.

8.2.3 External validity — where do these results transfer?

Every number comes from CUAD: **510 US commercial contracts drawn from SEC filings, in English**. We have not evaluated a single contract from outside that corpus. Specifically we have no evidence about: other drafting traditions or jurisdictions, including **Bangladeshi contracts**, which is the deployment context motivating this work; consumer or employment agreements, which differ in structure and in who the weaker party is; non-English contracts; contracts shorter or far longer than CUAD’s distribution; and contemporary machine-drafted agreements. The risk taxonomy carries an additional jurisdictional threat: a non-compete’s enforceability varies by jurisdiction, so a level that is right in one place may be wrong in another. CUAD’s SEC provenance also biases toward large-corporate agreements between represented parties, which is close to the opposite of the small-business and individual signers the system is aimed at.

8.2.4 Statistical conclusion validity are the intervals trustworthy?

Three sources of variance were measured, and they disagree in magnitude: the cluster bootstrap over contracts gives an interval half-width of about 0.008 (Section 6.1.2); model-seed SD is 0.0022 over three runs (Section 6.1.3); dataset-split SD is 0.0063 over twenty splits (Section 6.1.6). All three exceed the ensemble’s +0.0014 margin, which is why we call it a tie with some confidence. Split variance is the largest, and it is the one we could measure only for the baseline.

Three seeds is a small sample from which to estimate a standard deviation; the interval around 0.0022 is itself wide.

Rare-category statistics are not meaningful individually. Five categories have at most seven test positives. Their per-category F1 values are reported for completeness, not as estimates, and no claim in this thesis rests on any of them individually.

One split, one corpus. The bootstrap resamples contracts within our test set; it cannot represent variation across corpora, and no interval in this thesis should be read as covering that.

8.3 Limitations

Section 8.2 analysed the threats to each kind of validity. This section lists the limitations of the artefact itself what the system does not do, as distinct from what could undermine the measurements. Both are stated where they arise in the text; these are pointers.

No usability evaluation (Section 7.7). Nobody outside the three of us has used Clausify. The thesis argues that a risk-prioritized report is more useful to a non-lawyer than a list of clauses, and offers no evidence for it: no task-completion measurement, no comprehension check, no trust or over-reliance measurement. Given that Section 6.2 shows the confidence display is miscalibrated and Section 6.5 shows the plain-English sentence is category-generic, over-reliance is the specific thing a usability study would need to look for.

Backbone scale. We downsized the backbones (DistilBERT, FLAN-T5-small) to fit a laptop compute budget, so our absolute scores understate what the same method would achieve with larger backbones on CUDA hardware.

Single-seed and single-budget training, partly addressed. Section 6.1.4 reports what happened when the presence transformer was retrained on all 53,662 window examples instead of the 24,000-example subsample, and Section 6.1.3 reports the spread across three training seeds. The other two models span and summarizer are still single runs at a subsampled budget (8,000 of 11,156 and 4,000 of 11,156), and we have not measured their seed variance at all. Their reported scores should be read with that in mind.

The rare tail is close to unlearnable at this dataset size, and Section 6.1.10 shows the conjunctive ensemble makes it worse rather than better: on the ten rarest categories the AND rule nets no F1 gain over the transformer alone, buying precision entirely at the cost of recall, and it zeroes out three categories that the transformer alone was partly finding. Since one of those rare categories is High-risk in our taxonomy, this is a limitation of the deployed system and not only of the leaderboard.

Span evaluation scope. Section 6.4 measures extraction within answer-bearing windows, not the heavier full-document precision-at-recall benchmark. Section 6.6 closes part of that gap by

measuring what survives the real pipeline, but it is not the same protocol as the CUAD paper’s and we do not present it as comparable.

The summarizer does not summarize. Section 6.5 establishes this quantitatively rather than as a caveat: the fine-tuned model emits one of 41 pre-written templates verbatim on every input, and against clause-specific references it scores no better than the bare template with no model at all. It is a classification-to-template component and should be described as one.

The risk taxonomy has not been externally validated. It was written by three computer science students, and as of this writing no legal professional has reviewed it (Section 4.5.4). It is also per-category rather than per-instance it flags a category’s typical danger, not the severity of the specific wording in front of the user.

One dataset, one jurisdiction. Every number in this thesis comes from CUAD: 510 US commercial contracts drawn from SEC filings. We have not tested the pipeline on a single contract from outside that corpus, so we have no evidence about how it behaves on other drafting conventions, and none at all about the Bangladeshi deployment context that motivated the work.

8.4 Future Work

training contracts, select the ensemble rule and per-model thresholds on it, and report once on the test set. Section 6.3 suggests OR would win; choosing it on the evidence we have would be test-set selection.

- **Calibrate the confidence score.** Platt scaling or isotonic regression on a validation split would make the application's confidence bar mean what a user reads it to mean (ECE 0.201 at present). No retraining required.
- **Give the span model a way to abstain.** SQuAD v2-style unanswerable examples would stop it answering confidently from windows that cannot contain the clause one of the two causes of the 21.7% end-to-end rate (Section 6.6).
- **Train the span model on the windows it will actually see.** Varying the answer offset during training, instead of re-centring every example 150 characters in, addresses the other cause.
- **Evaluate summaries for faithfulness, not overlap.** BERTScore, an entailment-based consistency check, a readability statistic, and a small human rating of whether legal meaning survives (Section 2.7).
- **Attention pooling over windows.** The ablation in Section 6.1.5 varied only fixed statistics; a learned aggregator that weights windows by relevance is the untested option most likely to help.
- **Cross-validation rather than one split.** Dataset-split SD is the largest uncertainty we measured (Section 6.1.6) and we could only measure it for the baseline.
- **A genuinely abstractive summarizer.** Grow the 41-template seed into a clause-diverse target set and re-fine-tune. The 150-clause pilot in Section 6.5 is a working prototype of the annotation protocol and of the evaluation that would detect success; scaling it to a few thousand clauses is the actual path from template classification to simplification.
- **Risk-aware ensemble rules.** Section 6.1.10 suggests, but does not test, applying the permissive (OR) rule on High-risk categories and the conjunction elsewhere, so that recall is protected exactly where a missed clause costs the signer most. This needs its own held-

out evaluation choosing the rule after seeing the test set would invalidate it.

- **Dense retrieval.** Our 64% retrieval ceiling (Section 4.2.2) was measured with TF-IDF query scoring only. A sentence-encoder retriever matches on meaning rather than shared vocabulary and may raise that ceiling enough to make a retrieve-then-read pipeline competitive; the measurement is cheap and we should have run it.
- **A trained sparse-attention baseline.** Section 2.3 argues against Longformer on grounds that are partly arithmetic and partly cost, and Section 5.7 measures the cost but nobody has actually trained one on this task. On CUDA hardware, a Longformer-with-windowing baseline is the honest comparison our argument currently substitutes reasoning for.
- **Scale the backbones.** Re-run the unchanged pipeline with DeBERTa-v3-base/large and FLAN-T5-base on CUDA hardware to close the gap to the published CUAD baselines.
- **Full-document span benchmark.** Implement the document-stride ranked evaluation so we can report precision at 80% recall and compare with the CUAD paper apples-to-apples.
- **Cross-clause conflict detection.** Bring the rule engine designed in our pre-thesis phase (for example, *Exclusivity* vs. *Most Favored Nation*) into the deployed application.
- **Instance-level risk.** Move from per-category risk to per-clause severity by reading the extracted span itself a cap’s amount, a non-compete’s duration.
- **Contracts from outside CUAD.** Even three locally sourced English-language commercial contracts, run through the pipeline and inspected by hand, would tell us more about generalization than any further tuning on CUAD, and it is the first step towards the Bangladeshi deployment this work was aimed at.

Bibliography

- [1] D. Hendrycks, C. Burns, A. Chen, and S. Ball, “CUAD: An expert-annotated NLP dataset for legal contract review,” in *Proc. 35th Conf. Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [2] The Atticus Project, “Contract Understanding Atticus Dataset (CUAD) v1,” 2021. [Online]. Available: <https://www.atticusprojectai.org/cuad>
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5998–6008.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” in *5th Workshop on Energy Efficient Machine Learning and Cognitive Computing (NeurIPS)*, 2019.
- [6] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [7] H. W. Chung *et al.*, “Scaling instruction-finetuned language models,” *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.
- [8] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proc. EMNLP*, 2016, pp. 2383–2392.
- [9] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv preprint arXiv:2004.05150*, 2020.

- [10] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, “LEGAL-BERT: The muppets straight out of law school,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 2898–2904.
- [11] I. Chalkidis, A. Jana, D. Hartung, M. Bommarito, I. Androutsopoulos, D. M. Katz, and N. Aletras, “LexGLUE: A benchmark dataset for legal language understanding in English,” in *Proc. 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [12] L. Manor and J. J. Li, “Plain English summarization of contracts,” in *Proc. Natural Legal Language Processing Workshop (NLLP)*, 2019, pp. 1–11.
- [13] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On faithfulness and factuality in abstractive summarization,” in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 1906–1919.
- [14] W. Kryściński, B. McCann, C. Xiong, and R. Socher, “Evaluating the factual consistency of abstractive text summarization,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 9332–9346.
- [15] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *Int. Conf. Learning Representations (ICLR)*, 2020.
- [16] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.
- [17] O. Maron and T. Lozano-Pérez, “A framework for multiple-instance learning,” in *Advances in Neural Information Processing Systems 10 (NIPS)*, 1998, pp. 570–576.
- [18] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [19] R. Flesch, “A new readability yardstick,” *Journal of Applied Psychology*, vol. 32, no. 3, pp. 221–233, 1948.
- [20] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out (ACL Workshop)*, 2004, pp. 74–81.

Appendix A

Full Risk Taxonomy

The complete 41-category risk taxonomy (8 High, 22 Medium, 11 Low), scored from the weaker party's perspective, with the one-line reason shown to users in the Clausify application.

Table A.1: The 41 CUAD clause categories with risk level and reason.

Category	Risk	Reason
Non-Compete	High	Limits your future income and freedom to work.
Uncapped Liability	High	Unlimited damages — no ceiling on what you can owe.
IP Ownership Assignment	High	You lose ownership of a core asset for good.
Exclusivity	High	Locks you to a single source or buyer.
Most Favored Nation	High	You must always give the other side your best terms.
Liquidated Damages	High	Fixed penalties trigger automatically on breach.
Irrevocable or Perpetual License	High	A grant you can never take back.
Minimum Commitment	High	You are forced to buy or deliver a minimum amount.
Renewal Term	Medium	Auto-renewal can extend the contract unexpectedly.
Notice Period to Terminate Renewal	Medium	Miss the window and you are locked in for another term.

Category		Risk	Reason
Competitive Restriction		Medium	Carve-outs to competitive limits — read carefully.
Exception			
No-Solicit of Customers		Medium	Restricts approaching the other side's customers.
No-Solicit of Employees		Medium	Restricts hiring the other side's staff.
Non-Disparagement		Medium	Limits what you can publicly say.
Termination for Convenience		Medium	The other side can walk away with notice.
ROFR / ROFO / ROFN		Medium	First-refusal rights can constrain your options.
Change of Control		Medium	A sale or merger may trigger termination.
Anti-Assignment		Medium	You cannot transfer the contract without consent.
Revenue/Profit Sharing		Medium	You must share a slice of revenue or profit.
Price Restrictions		Medium	Limits what prices you may set or change.
Volume Restriction		Medium	Caps how much you can sell or distribute.
Joint IP Ownership		Medium	IP is co-owned — control is shared, not sole.
Non-Transferable License		Medium	The licence cannot be passed on.
Affiliate License (Licensor)		Medium	Licensor's affiliates are pulled into the grant.
Unlimited / All-You-Can-Eat License		Medium	Broad, unlimited licence scope — check who benefits.
Source Code Escrow		Medium	Source is held by a third party under conditions.
Post-Termination Services		Medium	Obligations continue after the contract ends.
Audit Rights		Medium	The other side can inspect your records.
Cap on Liability		Medium	Caps recovery — may under-compensate real losses.
Covenant Not to Sue		Medium	You waive the right to sue over listed matters.

Category	Risk	Reason
Document Name	Low	Identifies the agreement — informational.
Parties	Low	Names who is signing — informational.
Agreement Date	Low	The signing date — informational.
Effective Date	Low	When terms start — informational.
Expiration Date	Low	When the contract ends — informational.
Governing Law	Low	Which jurisdiction's law applies — neutral.
License Grant	Low	You receive usage rights — generally favorable.
Warranty Duration	Low	States how long the warranty protects you.
Insurance	Low	Required coverage — a protection.
Affiliate License (Licensee)	Low	Extends rights to your affiliates — favorable.
Third Party Beneficiary	Low	Names an outside beneficiary — informational.

Appendix B

Reproducibility Checklist

Everything below describes the environment in which every number in this thesis was produced. The purpose is that a reader with the repository and this appendix can regenerate any table without asking us a question.

Environment

Python	3.13.5
Operating system	macOS 26.5.2, arm64
Hardware	Apple M4, 10 cores (4 performance / 6 efficiency), 16 GB unified memory
Accelerator	Metal Performance Shaders (MPS); no CUDA at any point

torch	2.8.0
transformers	5.12.1
datasets	5.0.0
scikit-learn	1.6.1
numpy	2.1.3
pandas	2.2.3
rouge_score	installed (ROUGE-L is additionally implemented in- line; see below)
matplotlib	figures only

Data

Dataset	CUAD v1, Hugging Face theatticusproject/cuad
Files used	CUAD_v1/CUAD_v1.json (contexts, character offsets); CUAD_v1/master_clauses.csv (exploratory analysis only)
Contracts	510; 41 categories; 20,910 (contract, category) rows
Note	The repository's default <code>load_dataset()</code> configuration exposes only the raw PDFs and cannot be trained on (Section 3.1).

Seeds and splits

- Global seed 42 for the split, all three trainings, and every bootstrap.
- The split is *drawn once* from `numpy.random.default_rng(42).permutation`, saved to `artifacts/split.json`, and loaded from disk by every subsequent script — so no script re-derives it.
- Additional presence-model seeds 43 and 44 (Section 6.1.3); dataset-split seeds 0–19 for Section 6.1.6.
- Bootstrap: 10,000 resamples, `default_rng(42)`, resampling unit = contract.

Artifact fingerprints

First 16 hex characters of the SHA-256 of each artifact, so a reader can confirm they hold the same files:

CUAD_v1.json	ed0b77d85bdf4014
artifacts/split.json	177e2ef48dfab9cd
artifacts/baseline.pkl	1c26214794393bdb
outputs/presence_mil/final/model.safetensors	454a6c3c43aeea9
outputs/span/final/model.safetensors	f97ba9b172beac42
outputs/summarizer/final/model.safetensors	9fcb6bdf777d033c

Commands, and which table each regenerates

Command	Regenerates
python notebooks/build_artifacts.py	All three checkpoints, the split and the fitted baseline (≈ 85 min)
jupyter run notebooks/test.ipynb	Tables 6.1–6.5 (presence, confusion matrix, span, summarizer)
python analysis/bootstrap_ci.py	Table 6.2 (cluster bootstrap)
python analysis/presence_run.py --tag full53k --max-train 0	Table 6.4 (full-data control, ≈ 140 min train + 50 min eval)
python analysis/presence_run.py --tag s43 --seed 43	Table 6.3 (seed variance; repeat for 44)
python analysis/compare_runs.py	Tables 6.4 and 6.3 contrasts
python analysis/span_ci.py	Table 6.12 (span intervals)
python analysis/rare_tail.py	Table 6.10
python analysis/risk_threshold_calib.py	Table 6.11, Figure 6.4
python analysis/split_variance.py	Table 6.6
python analysis/e2e.py AND	Tables 6.15 and 6.16
python analysis/summ_pilot.py	Table 6.13
python analysis/error_examples.py	Table 6.7
python analysis/lf_one.py	Table 5.3 (one row per invocation)
longformer 4096 1 train	
python analysis/app_bench.py	Table 7.1
python analysis/verify_numbers.py	Cross-checks all 84 reported figures and 17 derived comparisons in Chapters 5–8 against

Known determinism caveats

- MPS kernels are not bit-reproducible across torch versions; a rerun should reproduce reported scores to roughly three decimal places, not exactly.
- ROUGE-L in `summ_pilot.py` is computed by an inline longest-common-subsequence implementation (matching `test.ipynb`) rather than by the `rouge_score` package, so the two are not interchangeable.
- The retrieval-ceiling measurement of Section 4.2.2 was computed over all 510 contracts before the split was drawn (Section 8.2).

Appendix C

Repository and Artifact Layout

```
clausify-research/                                -- research deliverable
|- PROJECT_REPORT.md / .pdf                       -- step-by-step project report
|- REPORT.md / .pdf                               -- findings & caveats report
|- METHODOLOGY.md / .pdf                         -- design-decision record
|- data/
|   |- train_80.csv                               -- 80% split (16,728 rows, 408 contracts)
|   |- test_20.csv                                -- 20% split (4,182 rows, 102 contracts)
|- notebooks/
|   |- train.ipynb                                -- training notebook (EDA -> 3 models)
|   |- test.ipynb                                 -- STANDALONE evaluation notebook
|   |- eda.ipynb                                  -- original combined notebook
|   |- build_artifacts.py                         -- retrains & saves all artifacts
|   |- artifacts/                                 -- split.json, baseline.pkl
|   |- outputs/                                   -- three trained model checkpoints
|- review/
|   |- risk_taxonomy_review_sheet.csv             -- face-validity review instrument
|   |- risk_taxonomy_review_sheet.md              -- printable version with instructions
|- analysis/
|   |- bootstrap_ci.py                            -- cluster bootstrap, presence models
|   |- span_ci.py                                 -- cluster bootstrap, span extraction
|   |- rare_tail.py                               -- rare-category AND-rule breakdown
|   |- e2e.py                                     -- end-to-end pipeline evaluation
|   |- summ_pilot.py                              -- two-reference-set summarizer pilot
|   |- presence_run.py                            -- presence training at any data budget
|   |- longformer_bench.py                       -- sparse-attention cost benchmark
```

```
| |- diverse_refs.json          -- 150 clause-specific pilot references
|- thesis/                     -- this document (LaTeX)

clausify-app/                  -- Clausify demo application
|- app.py                      -- Streamlit UI
|- model_utils.py              -- inference + risk taxonomy
|- cat_questions.json          -- 41 category->question strings
|- sample_contract.txt         -- real CUAD test contract
|- test_contract.txt           -- synthetic multi-clause contract
|- APP_REPORT.md / .pdf       -- application report
```